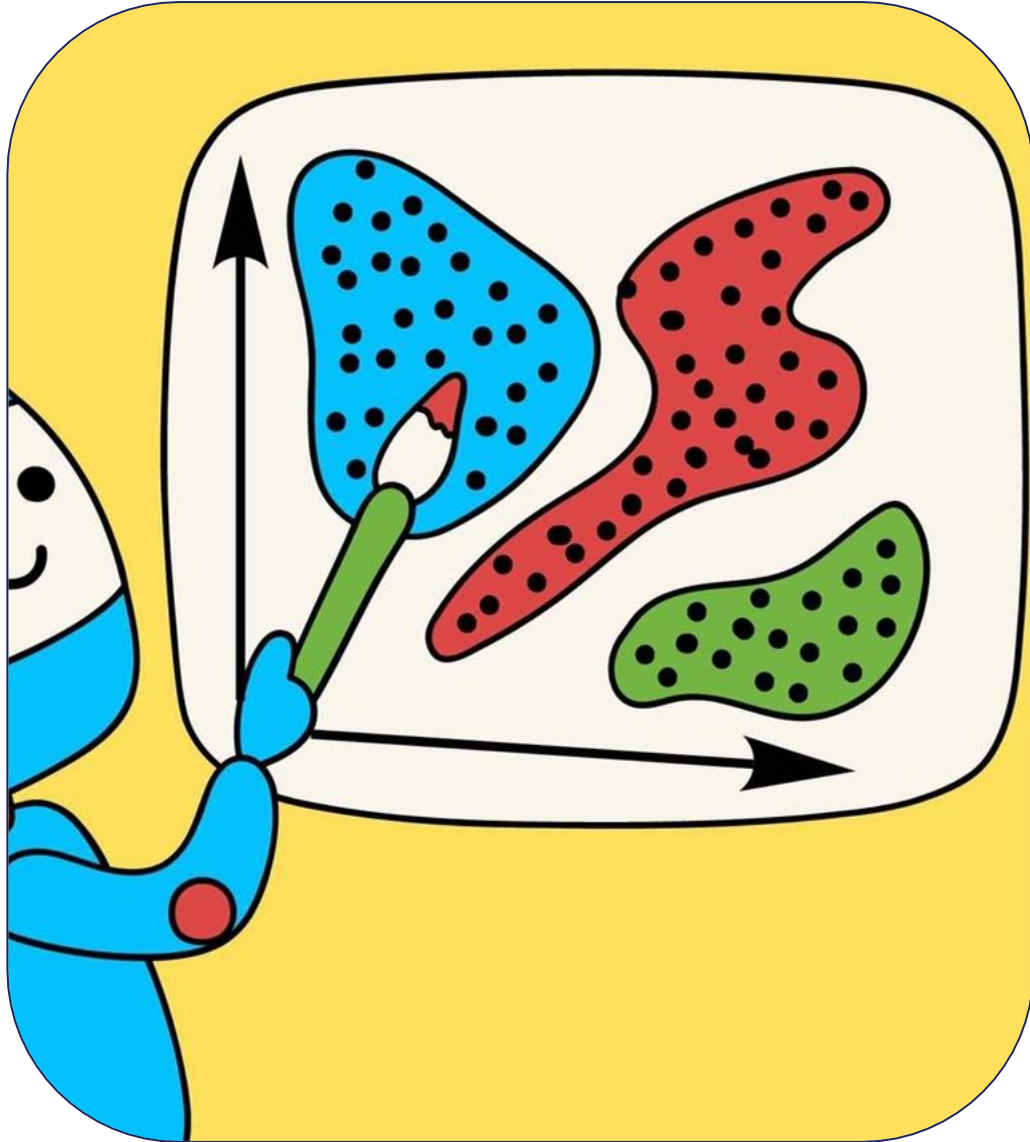




05

Cluster Analysis

Introduction

- Many applications require the partitioning of data points into intuitively similar groups.
- The partitioning of a large number of data points into a smaller number of groups helps greatly in summarizing the data and understanding it for a variety of data mining applications.
- An informal and intuitive definition of clustering is as follows:

Given a set of data points, partition them into groups containing very similar data points.

- This is a very rough and intuitive definition because it does not state much about the different ways in which the problem can be formulated, such as the number of groups, or the objective criteria for similarity. Nevertheless, this simple description serves as the basis for a number of models that are specifically tailored for different applications.

Introduction

Examples of applications:

- *Data summarization*: At the broadest level, the clustering problem may be considered as a form of data summarization. As data mining is all about extracting summary information (or concise *insights*) from data, the clustering process is often the first step in many data mining algorithms. In fact, many applications use the summarization property of cluster analysis in one form or the other.
- *Customer segmentation*: It is often desired to analyze the common behaviors of groups of similar customers. This is achieved by *customer segmentation*. An example of an application of customer segmentation is *collaborative filtering*, in which the stated or derived preferences of a similar group of customers are used to make product recommendations within the group.

Introduction

- *Social network analysis*: In the case of network data, nodes that are tightly clustered together by linkage relationships are often similar groups of friends, or *communities*. The problem of community detection is one of the most widely studied in social network analysis, because a broader understanding of human behaviors is obtained from an analysis of community group dynamics.
- *Relationship to other data mining problems*: Due to the summarized representation it provides, the clustering problem is useful for enabling other data mining problems. For example, clustering is often used as a preprocessing step in many classification and outlier detection models.

Introduction

- A wide variety of models have been developed for cluster analysis. These different models may work better in different scenarios and data types.
- A problem, which is encountered by many clustering algorithms, is that many features may be **noisy** or **uninformative** for cluster analysis.
- Such features need to be removed from the analysis early in the clustering process.
- This problem is referred to as *feature selection*.

Introduction

- Our study of clustering will be restricted to simpler multidimensional data types, such as numeric or discrete data.
- The key models differ primarily in terms of **how similarity is defined within the groups of data**. In some cases, similarity is defined **explicitly** with an appropriate distance measure, whereas in other cases, it is defined **implicitly** with a probabilistic model or a density-based model. In addition, certain scenarios for cluster analysis, such as high-dimensional or very large-scale data sets, pose special challenges.

Feature Selection for Clustering

- The key goal of feature selection is **to remove the noisy attributes that do not cluster well**.
- Feature selection is generally **more difficult** for *unsupervised* problems, such as clustering, where external validation criteria, such as labels, are not available for feature selection.
- Intuitively, the problem of feature selection is intimately related to that of determining the inherent *clustering tendency* of a set of features. Feature selection methods determine subsets of features that maximize the underlying clustering tendency.

Feature Selection for Clustering

- There are two primary classes of models for performing feature selection:
 1. *Filter models*: In this case, a score is associated with each feature with the use of a **similarity-based criterion**. This criterion is essentially a *filter* that provides a crisp condition for feature removal. Data points that do not meet the required score are removed from consideration. In some cases, these models may quantify the quality of a subset of features as a *combination*, rather than a single feature. Such models are more powerful because they implicitly take into account the *incremental* impact of adding a feature to others.

Feature Selection for Clustering

2. *Wrapper models*: In this case, a clustering algorithm is used to evaluate the quality of a subset of features. This is then used to refine the subset of features on which the clustering is performed. This is a naturally **iterative approach** in which a good choice of features depends on the clusters and vice versa. The features selected will typically be at least somewhat **dependent on the particular methodology used for clustering**. Although this may seem like a disadvantage, the fact is that different clustering methods may work better with different sets of features. Therefore, **this methodology can also optimize the feature selection to the specific clustering technique**. On the other hand, the inherent informativeness of the specific features may sometimes not be reflected by this approach due to the impact of the specific clustering methodology.

Representative-Based Algorithms

- Representative-based algorithms are the simplest of all clustering algorithms because they rely directly on intuitive notions of **distance** (or **similarity**) to cluster data points.
- In representative-based algorithms, the clusters are created in one shot, and hierarchical relationships do not exist among different clusters. This is typically done with the use of a set of **partitioning representatives**. The partitioning representatives may either be created as a function of the data points in the clusters (e.g., the mean) or may be selected from the existing data points in the cluster.
- The main insight of these methods is that **the discovery of high-quality clusters in the data is equivalent to discovering a high-quality set of representatives**. Once the representatives have been determined, a distance function can be used to assign the data points to their closest representatives.

Representative-Based Algorithms

- Typically, it is assumed that **the number of clusters, denoted by k , is specified by the user**. Consider a data set $\mathcal{P}$ containing n data points denoted by $\underline{X}_1 \dots \underline{X}_n$ in d -dimensional space.
- The goal is to determine k representatives $\underline{Y}_1 \dots \underline{Y}_k$ that minimize the following objective function O :

$$O = \sum_{i=1}^n [\min_j \text{Dist}(\underline{X}_i, \underline{Y}_j)] .$$

- In other words, the sum of the distances of the different data points to their closest representatives needs to be minimized. Note that the assignment of data points to representatives depends on the choice of the representatives $\underline{Y}_1 \dots \underline{Y}_k$.
- In some variations of representative algorithms, such as ***k-medoid algorithms***, it is assumed that the representatives $\underline{Y}_1 \dots \underline{Y}_k$ are drawn from the **original database** $\mathcal{P}$, although this will obviously not provide an optimal solution. In general, **we will not automatically assume that the representatives are drawn from the original database $\mathcal{P}$** , unless specified otherwise.

Representative-Based Algorithms

- One observation about the formulation $O = \sum_{i=1}^n [\min_j \text{Dist}(\overline{X}_i, \overline{Y}_j)]$ is that the representatives $\underline{Y}_1 \dots \underline{Y}_k$ and the optimal assignment of data points to representatives are unknown a priori, but they depend on each other in a circular way.
- For example, if the optimal representatives are known, then the optimal assignment is easy to determine, and vice versa. Such optimization problems are solved with the use of an iterative approach where candidate representatives and candidate assignments are used to improve each other.
- Therefore, the generic k -representatives approach starts by initializing the k representatives S with the use of a straightforward heuristic (such as random sampling from the original data), and then refines the representatives and the clustering assignment, iteratively, as follows:

Representative-Based Algorithms

- **(Assign step)** Assign each data point to its closest representative in S using distance function $Dist(\cdot, \cdot)$, and denote the corresponding clusters by $C_1 \dots C_k$.
- **(Optimize step)** Determine the **optimal representative** $\bar{Y}_j$ for each cluster C_j that minimizes its *local* objective function $\sum_{\bar{X}_i \in C_j} [Dist(\bar{X}_i, \bar{Y}_j)]$
- It will be evident that this two-step procedure is very closely related to **generative models** of cluster analysis in the form of *expectation-maximization* algorithms.
- The second step of *local* optimization is simplified by this two-step iterative approach, because it no longer depends on an unknown assignment of data points to clusters as in the global optimization problem of $O = \sum_{i=1}^n [\min_j Dist(\bar{X}_i, \bar{Y}_j)]$.
- Typically, the **optimized representative** can be shown to be some central measure of the data points in the j th cluster C_j , and the precise measure depends on the choice of the distance function $[Dist(\bar{X}_i, \bar{Y}_j)]$

Representative-Based Algorithms

- In particular, for the case of the **Euclidean distance** and **cosine similarity functions**, it can be shown that the optimal centralized representative of each cluster is its **mean**.
- However, different distance functions may lead to a slightly different type of centralized representative, and these lead to different variations of this broader approach, such as the k -means and k -medians algorithms.
- Thus, the k -representative approach defines a *family* of algorithms, in which minor changes to the basic framework allow the use of different distance criteria.

Representative-Based Algorithms

- The generic framework for representative-based algorithms with an unspecified distance function is illustrated in the following pseudocode.

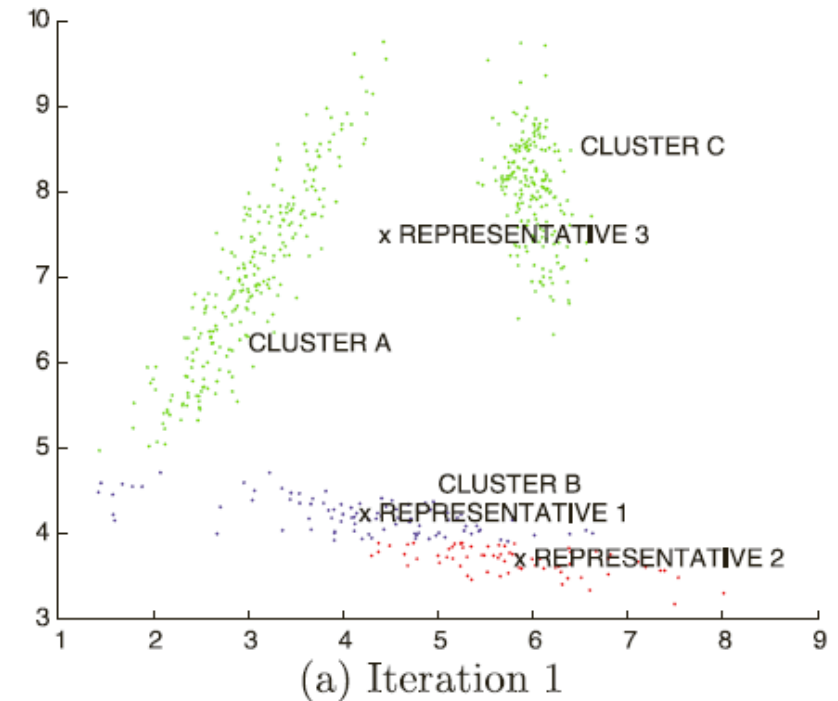
Algorithm *GenericRepresentative*(Database: $\mathcal{D}$, Number of Representatives: k)
begin
 Initialize representative set S ;
 repeat
 Create clusters $(\mathcal{C}_1 \dots \mathcal{C}_k)$ by assigning each
 point in $\mathcal{D}$ to closest representative in S
 using the distance function $Dist(\cdot, \cdot)$;
 Recreate set S by determining one representative $\bar{Y}_j$ for
 each $\mathcal{C}_j$ that minimizes $\sum_{\bar{X}_i \in \mathcal{C}_j} Dist(\bar{X}_i, \bar{Y}_j)$;
 until convergence;
 return $(\mathcal{C}_1 \dots \mathcal{C}_k)$;
end

Representative-Based Algorithms

- The idea is to improve the objective function over multiple iterations. Typically, **the increase is significant in early iterations, but it slows down in later iterations**. When the improvement in the objective function in an iteration is less than a user-defined threshold, the algorithm may be allowed to terminate.
- The primary **computational bottleneck** of the approach is the **assignment step** where the distances need to be computed between all point-representative pairs.

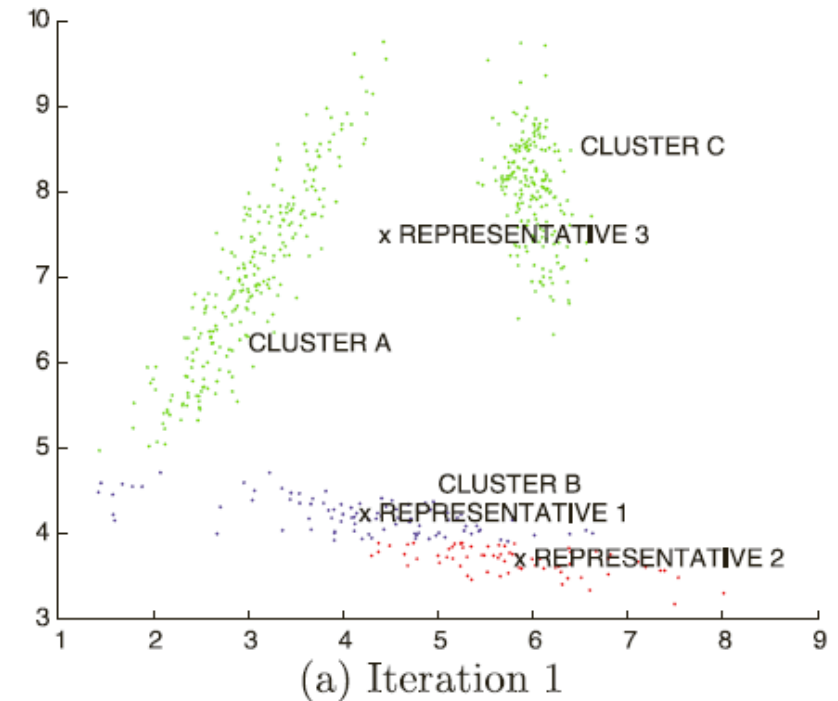
Representative-Based Algorithms

- Here the data contains three natural clusters, denoted by A, B, and C. For illustration, it is assumed that the input k to the algorithm is the same as the number of natural clusters in the data, which, in this case, is 3.
- The Euclidean distance function is used.



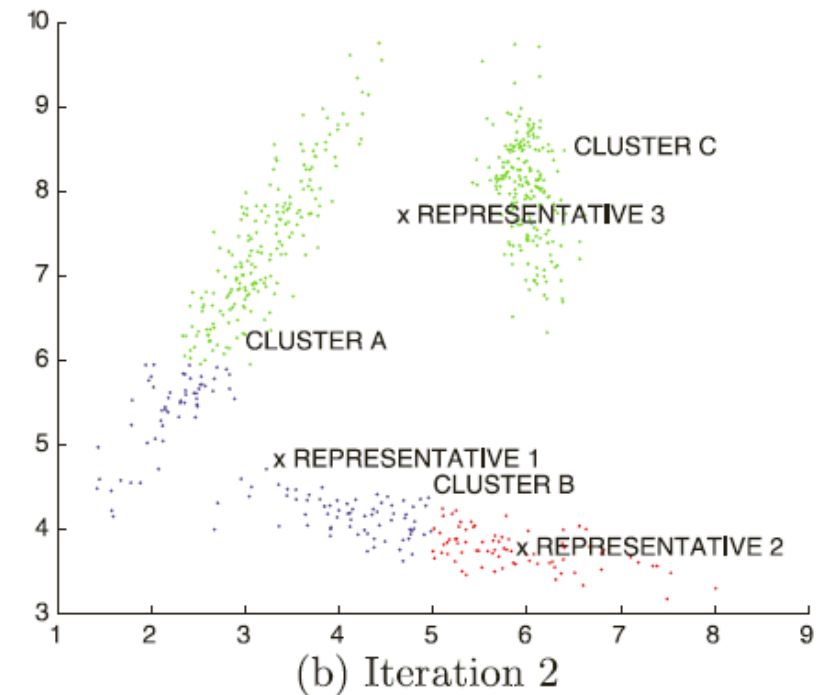
Representative-Based Algorithms

- Therefore the “re-centering” step uses the mean of the cluster. The initial set of representatives (or **seeds**) is chosen randomly from the data space.
- This leads to a particularly bad initialization, where two of the representatives are close to cluster B, and one of them lies somewhere midway between clusters A and C.
- As a result, the cluster B is initially split up by the “sphere of influence” of two representatives, whereas most of the points in clusters A and C are assigned to a single representative in the first assignment step.



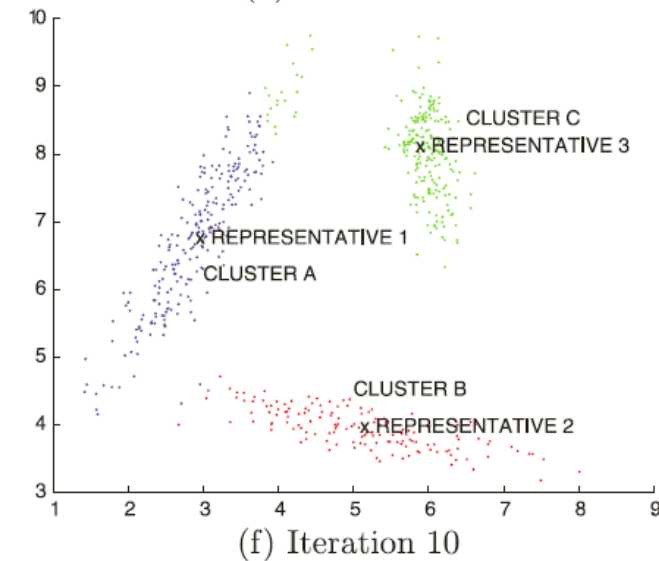
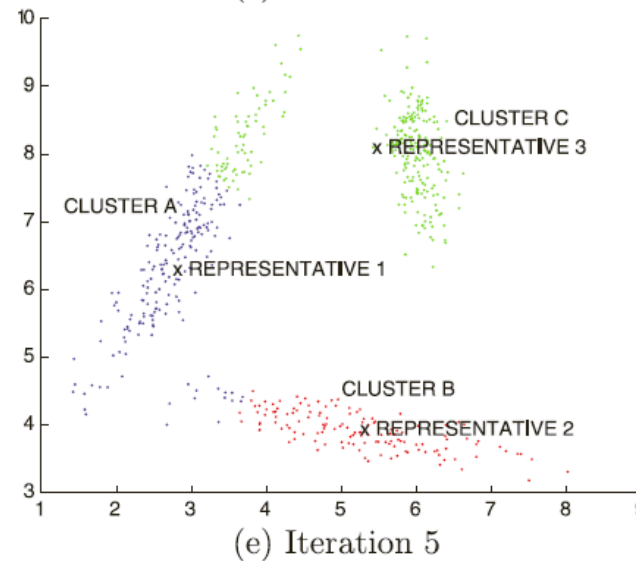
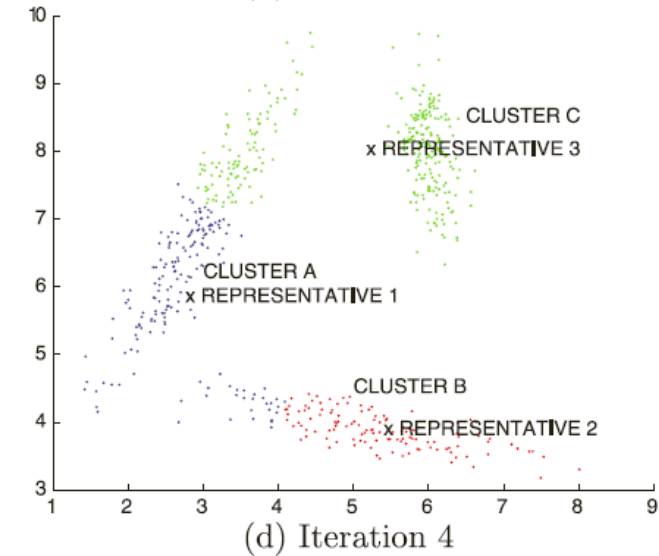
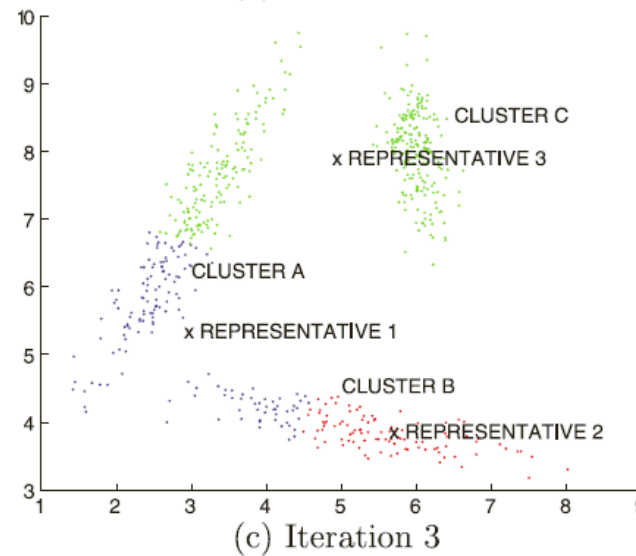
Representative-Based Algorithms

- However, because each representative is assigned a different number of data points from the different clusters, the representatives drift in subsequent iterations to one of the unique clusters.
- For example, representative 1 steadily drifts toward cluster A, and representative 3 steadily drifts toward cluster C.



Representative-Based Algorithms

- At the same time, representative 2 becomes a better centralized representative of cluster B. As a result, cluster B is no longer split up among different representatives by the end of iteration 10.



Representative-Based Algorithms

- An interesting observation is that even though the initialization was so poor, it required only 10 iterations for the k -representatives approach to create a reasonable clustering of the data. In practice, this is generally true of k -representative methods, which converge relatively fast toward a good clustering of the data points.
- However, it is possible for k -means to converge to **suboptimal** solutions, especially when an **outlier** data point is selected as an initial representative for the algorithm. In such a case, one of the clusters may contain a singleton point that is not representative of the data set, or it may contain two merged clusters.
- The handling of such cases is discussed later.
- We will discuss some special cases and variations of this framework. Most of the **variations** of the k -representative framework are defined by the choice of the distance function $Dist(\underline{X}_i, \underline{Y}_j)$ between the data points $\underline{X}_i$ and the representatives $\underline{Y}_j$. Each of these choices results in a different type of centralized representative of a cluster.

Representative-Based Algorithms

The k -Means Algorithm

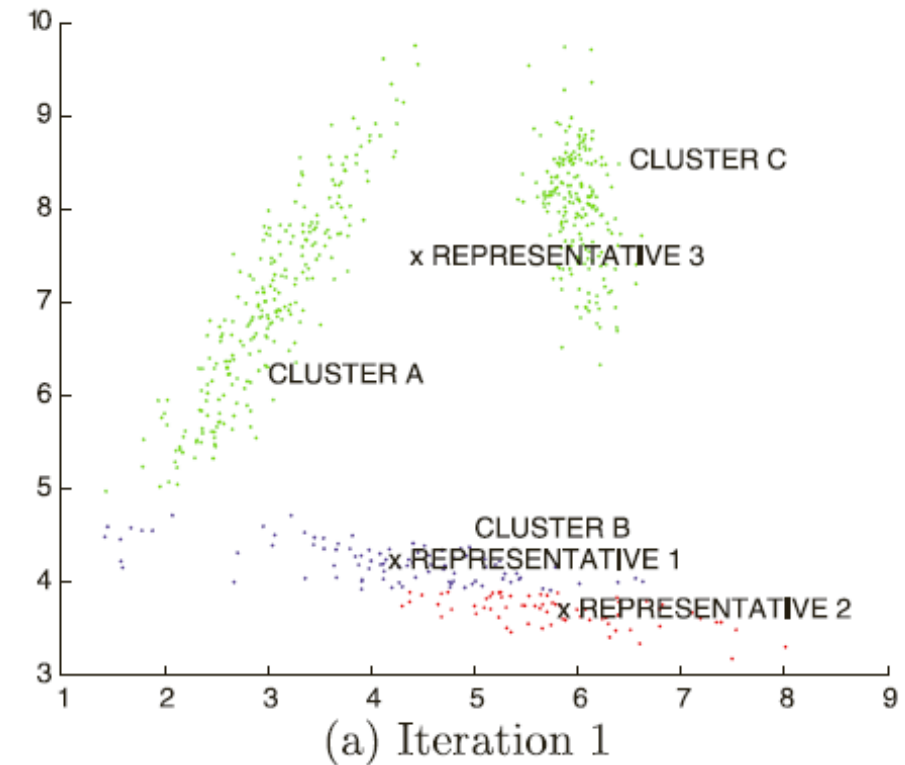
- In the **k-means algorithm**, the sum of the squares of the **Euclidean distances** of data points to their closest representatives is used to quantify the objective function of the clustering:

$$Dist(\overline{X_i}, \overline{Y_j}) = ||\overline{X_i} - \overline{Y_j}||_2^2$$

- Here, $|| \cdot ||_p$ represents the L_p -norm. The expression $Dist(\underline{X_i}, \underline{Y_j})$ can be viewed as the **squared error of approximating a data point with its closest representative**. Thus, the overall objective minimizes the sum of square errors over different data points. This is also sometimes referred to as **SSE**.
- In such a case, it can be demonstrated that *the optimal representative $\underline{Y_j}$ for each of the “optimize” iterative steps is the mean of the data points in cluster C_j .*

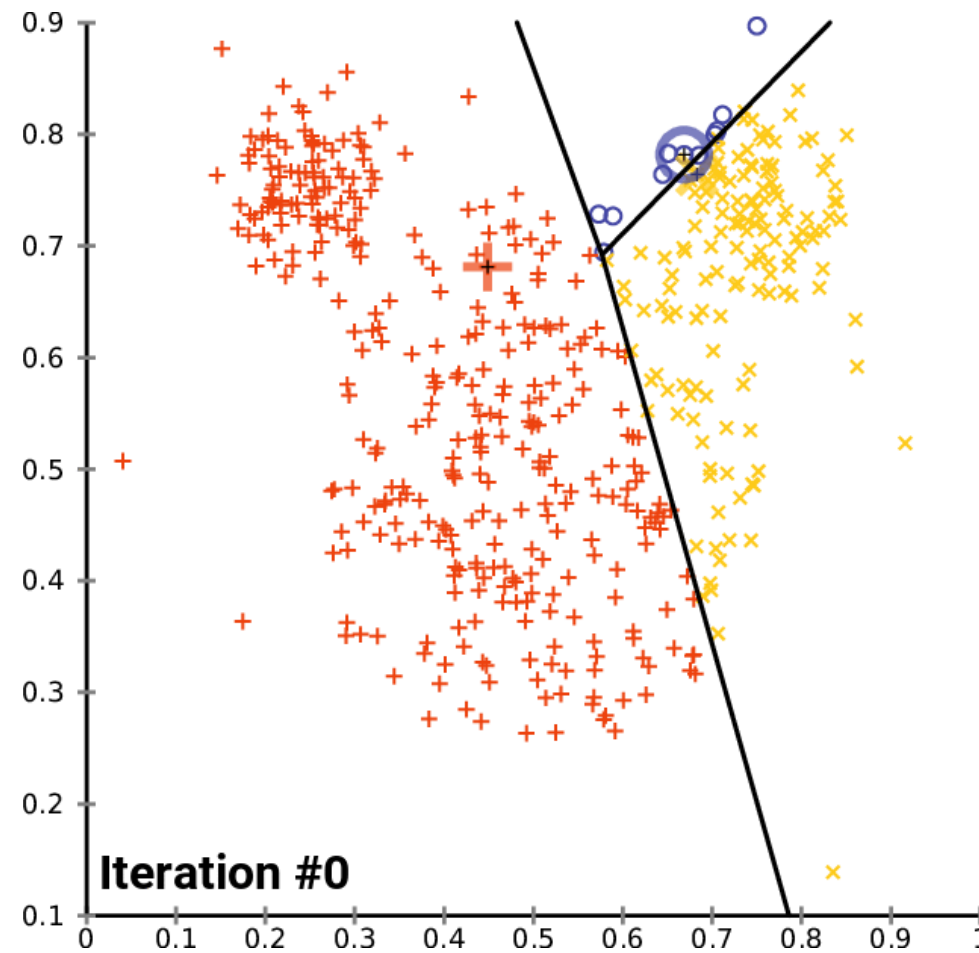
Representative-Based Algorithms

- An interesting variation of the k -means algorithm is to use the local **Mahalanobis distance** for assignment of data points to clusters.
- The use of the Mahalanobis distance is generally helpful **when the clusters are elliptically elongated along certain directions**, as in this case



Representative-Based Algorithms

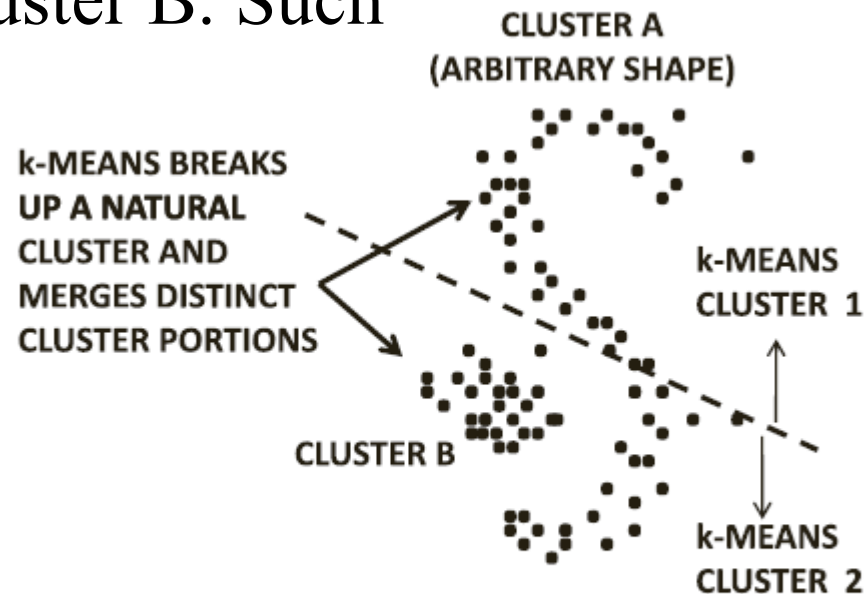
- Convergence of k -means



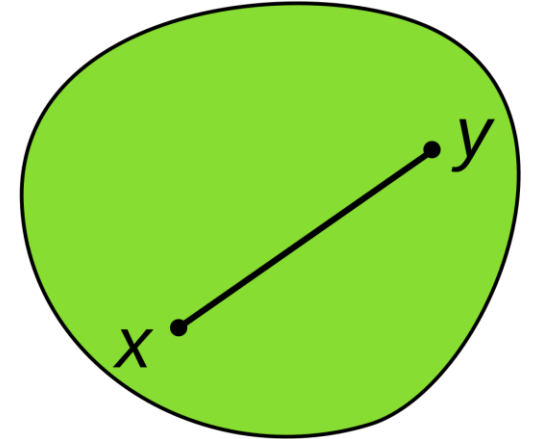
Representative-Based Algorithms

- The k -means algorithm does not work well when the clusters are of arbitrary shape.

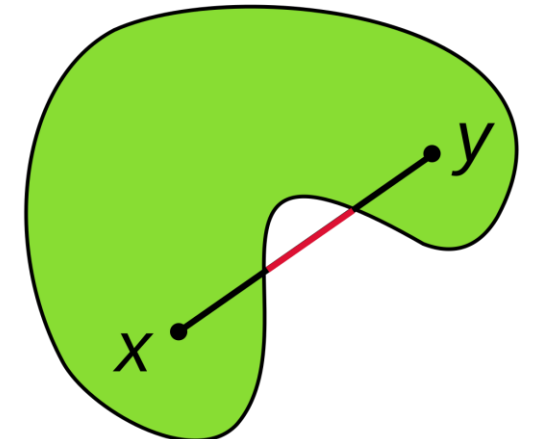
Example: cluster A has a nonconvex shape. The k -means algorithm breaks it up into two parts, and also merges one of these parts with cluster B. Such situations are common in k -means, because it is biased toward finding spherical clusters.



(a) Varying cluster shape
(Bad for k -means)



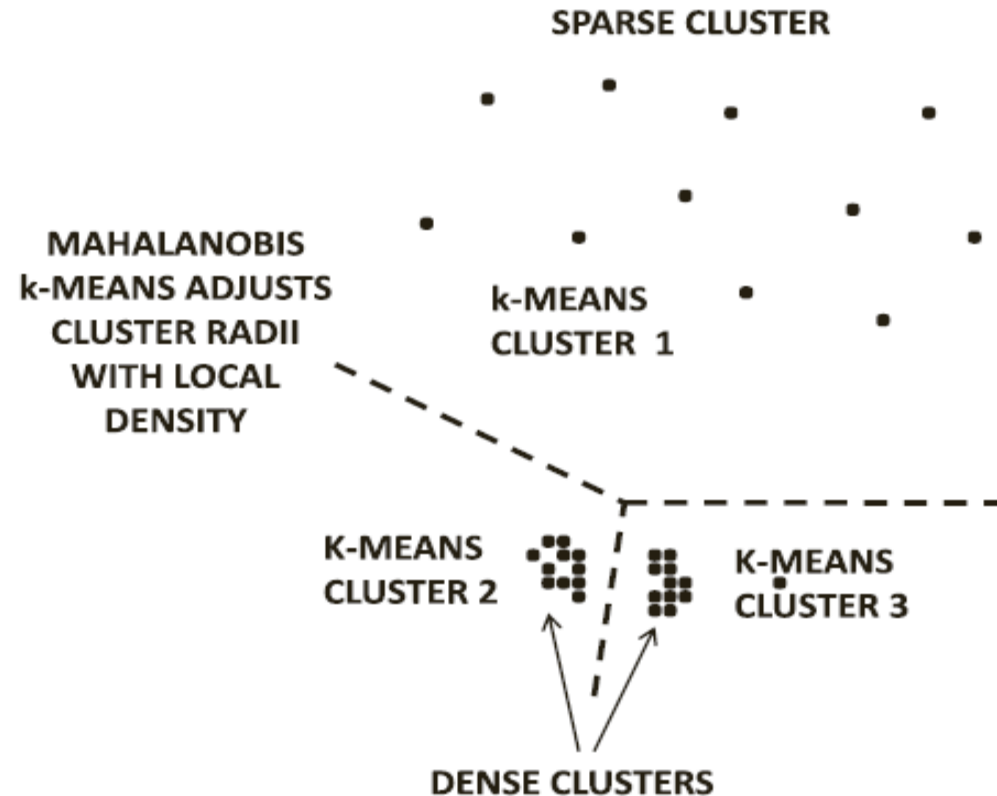
Cluster with convex shape



Cluster with nonconvex shape

Representative-Based Algorithms

- Even the Mahalanobis k -means algorithm does not work well in this scenario in spite of its ability to adjust for the elongation of clusters. On the other hand, the Mahalanobis k -means algorithm **can adjust well to varying cluster density**.

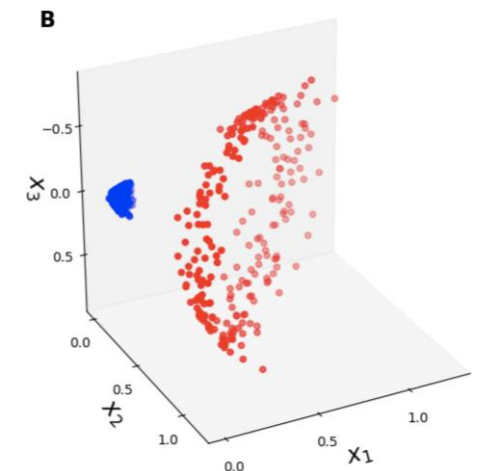
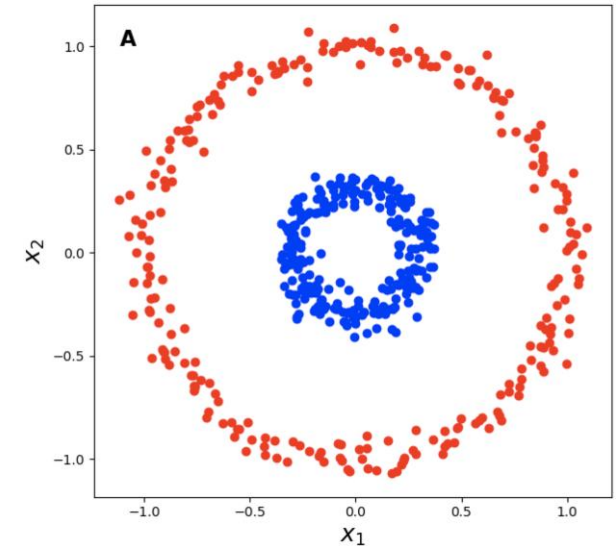


(b) Varying cluster density
(Good for Mahalanobis k -means)

Representative-Based Algorithms

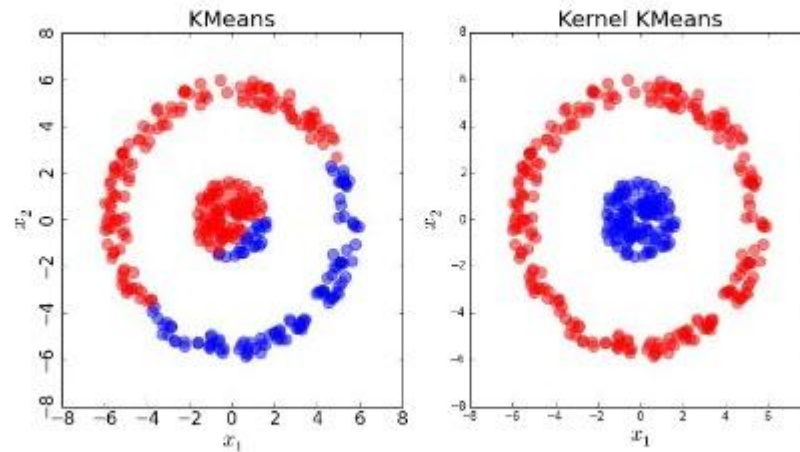
The Kernel k -Means Algorithm

- The k -means algorithm can be extended to discovering clusters of arbitrary shape with the use of a method known as the *kernel trick*. The basic idea is to implicitly transform the data so that arbitrarily shaped clusters map to Euclidean clusters in the new space.
- The main problem with the *kernel k -means algorithm* is that the complexity of computing the kernel matrix alone is quadratically related to the number of data points. Such an approach can effectively discover arbitrarily shaped clusters.



Representative-Based Algorithms

Kernel K-means vs. K-means



Pyclus: Open Source Data Clustering Package

Representative-Based Algorithms

The k -Medians Algorithm

- In the k -medians algorithm the Manhattan distance is used as the objective function of choice. Therefore, the distance function $Dist(\overline{X}_i, \overline{Y}_j)$ is defined as follows:

$$Dist(\overline{X}_i, \overline{Y}_j) = ||X_i - Y_j||_1$$

- In such a case, it can be shown that the optimal representative, $\overline{Y}_j$ is the median of the data points along each dimension in cluster C_j . This is because the point that has the minimum sum of L_1 -distances to a set of points distributed on a line is the median of that set.

Representative-Based Algorithms

Finding the median in sets of data with an odd and even number of values.

1, 3, 3, **6**, 7, 8, 9

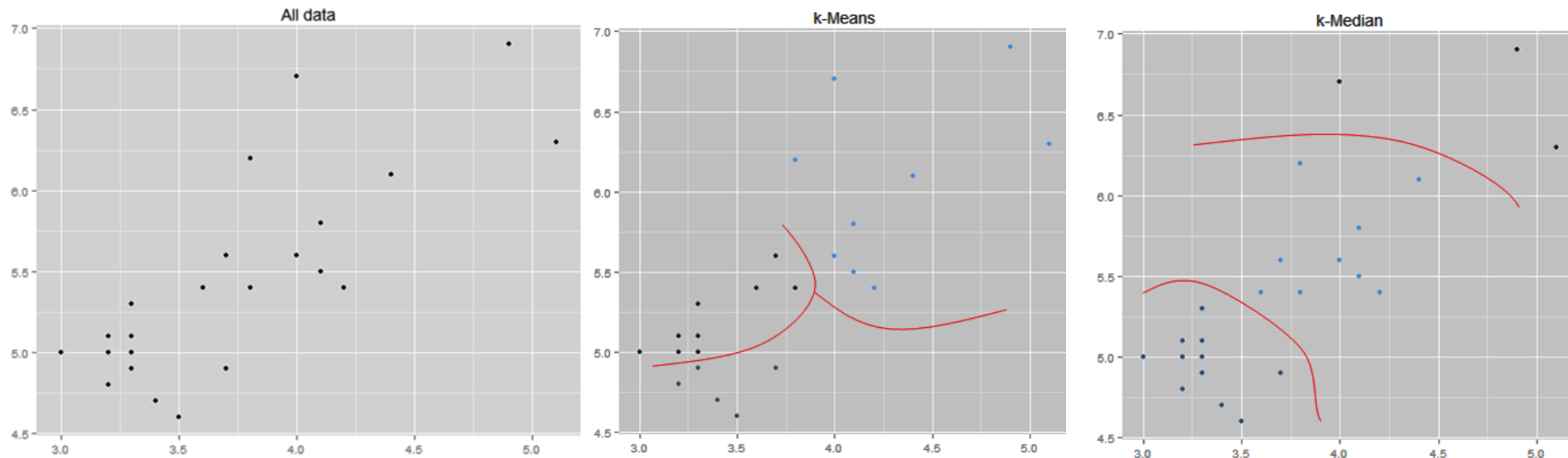
Median = **6**

1, 2, 3, **4**, **5**, 6, 8, 9

Median = $(4 + 5) \div 2$

= **4.5**

$k=3$



Representative-Based Algorithms

The k -Medoids Algorithm

- Although the k -medoids algorithm also uses the notion of representatives, its algorithmic structure is different from the generic k -representatives algorithm.
- The main distinguishing feature of the k -medoids algorithm is that the **representatives are always selected from the database $\mathcal{P}$** , and this difference necessitates changes to the basic structure of the k -representatives algorithm, although the clustering **objective function** $O = \sum_{i=1}^n [\min_j \text{Dist}(\overline{X_i}, \overline{Y_j})]$ is of the same form.

Representative-Based Algorithms

- Question: why is it sometimes desirable to select the representatives from $\mathcal{D}$?
- There are two reasons for this.
 1. The representative of a k -means cluster may be distorted by outliers in that cluster. In such cases, it is possible for the representative to be located in an empty region which is unrepresentative of most of the data points in that cluster. Such representatives may result in partial merging of different clusters, which is clearly undesirable. This problem can, however, be partially resolved with careful outlier handling and the use of outlier-robust variations such as the k -medians algorithm.

Representative-Based Algorithms

2. It is sometimes difficult to compute the optimal central representative of a set of data points of a complex data type. For example, if the k -representatives clustering algorithm were to be applied on a set of time series of *varying lengths*, then how should the central representatives be defined as a function of these heterogeneous time-series? In such cases, selecting representatives from the original data set may be very helpful. As long as a representative object is selected from each cluster, the approach will provide reasonably high quality results. Therefore, a key property of the k -medoids algorithm is that it can be defined virtually on any data type, as long as an appropriate similarity or distance function can be defined on the data type. Therefore, k -medoids methods directly relate the problem of distance function design to clustering.

Representative-Based Algorithms

- The k -medoids approach uses a generic hill-climbing strategy, in which the representative set $\mathcal{S}$ is initialized to **a set of points from the original database $\mathcal{P}$** .
- Subsequently, this set $\mathcal{S}$ is iteratively improved by **exchanging** a single point from set $\mathcal{S}$ with a data point selected from the database $\mathcal{P}$.
- This iterative exchange can be viewed as a hill-climbing strategy, because the set $\mathcal{S}$ implicitly defines a solution to the clustering problem, and **each exchange can be viewed as a hill-climbing step**.
- So what should be the criteria for the exchange, and when should one terminate?

Representative-Based Algorithms

- Clearly, in order for the clustering algorithm to be successful, the hill-climbing approach **should at least improve the objective function** of the problem to some extent. Several choices arise in terms of how the exchange can be performed:
 - One can try **all** $|\mathcal{S}| \cdot |\mathcal{P}|$ possibilities for replacing a representative in $\mathcal{S}$ with a data point in $\mathcal{P}$ and then select the best one. However, this is **extremely expensive** because the computation of the incremental objective function change for **each** of the $|\mathcal{S}| \cdot |\mathcal{P}|$ alternatives will require time proportional to the original database size.
 - A simpler solution is to use **a randomly selected set of r pairs**
 $(\overline{X_i}, \overline{Y_j})$
 for possible exchange, where $\underline{X_i}$ is selected from the database $\mathcal{P}$, and $\underline{Y_j}$ is selected from the representative set $\mathcal{S}$. The best of these r pairs is used for the exchange.

Representative-Based Algorithms

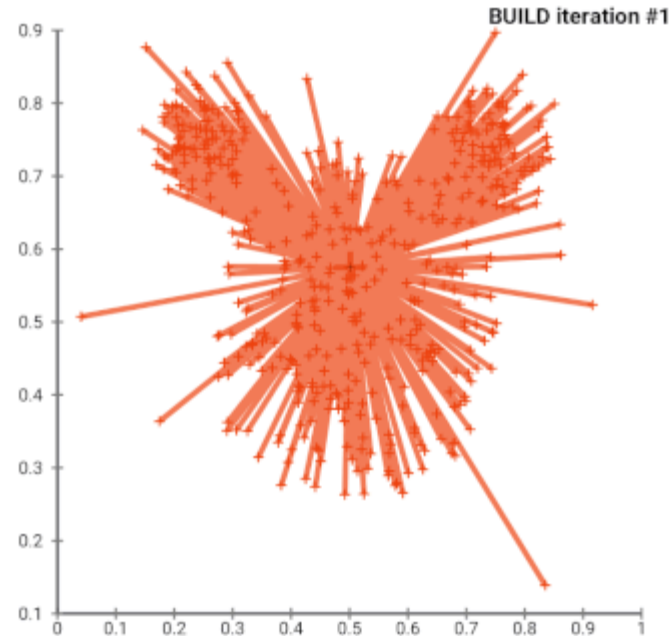
- The second solution requires time proportional to r times the database size but is usually practically implementable for databases of modest size.
- The solution is said to have converged when the objective function does not improve, or if the average objective function improvement is below a user-specified threshold in the previous iteration.
- The k -medoids approach is generally **much slower than the k -means** method but has greater applicability to different data types.

Representative-Based Algorithms

- The most common realisation of k -medoid clustering is the **partitioning around medoids (PAM)** algorithm.
- PAM uses a greedy search which may not find the optimum solution, but it is faster than exhaustive search. It works as follows:
 1. **Initialize:** greedily select k of the n data points as the medoids to minimize the cost
 2. **Associate** each data point to the closest medoid.
 3. **While** the cost of the configuration decreases:
 1. For each medoid m , and for each non-medoid data point o :
 1. Consider the swap of m and o , and compute the cost change
 2. If the cost change is the current best, remember this m and o combination in m_{best} and o_{best}
 2. Perform the best swap of m_{best} and o_{best} , if it decreases the cost function. Otherwise, the algorithm terminates.

Representative-Based Algorithms

PAM choosing initial medoids, then iterating to convergence for $k=3$ clusters.



Hierarchical Clustering Algorithms

- **Hierarchical algorithms** typically cluster the data with distances. However, the use of distance functions is not compulsory. Many hierarchical algorithms use other clustering methods, such as density- or graph-based methods, as a subroutine for constructing the hierarchy.
- So why are hierarchical clustering methods useful from an application-centric point of view?
- One major reason is that different levels of clustering granularity provide different application-specific insights. This provides a *taxonomy* of clusters, which may be browsed for semantic insights.

Hierarchical Clustering Algorithms

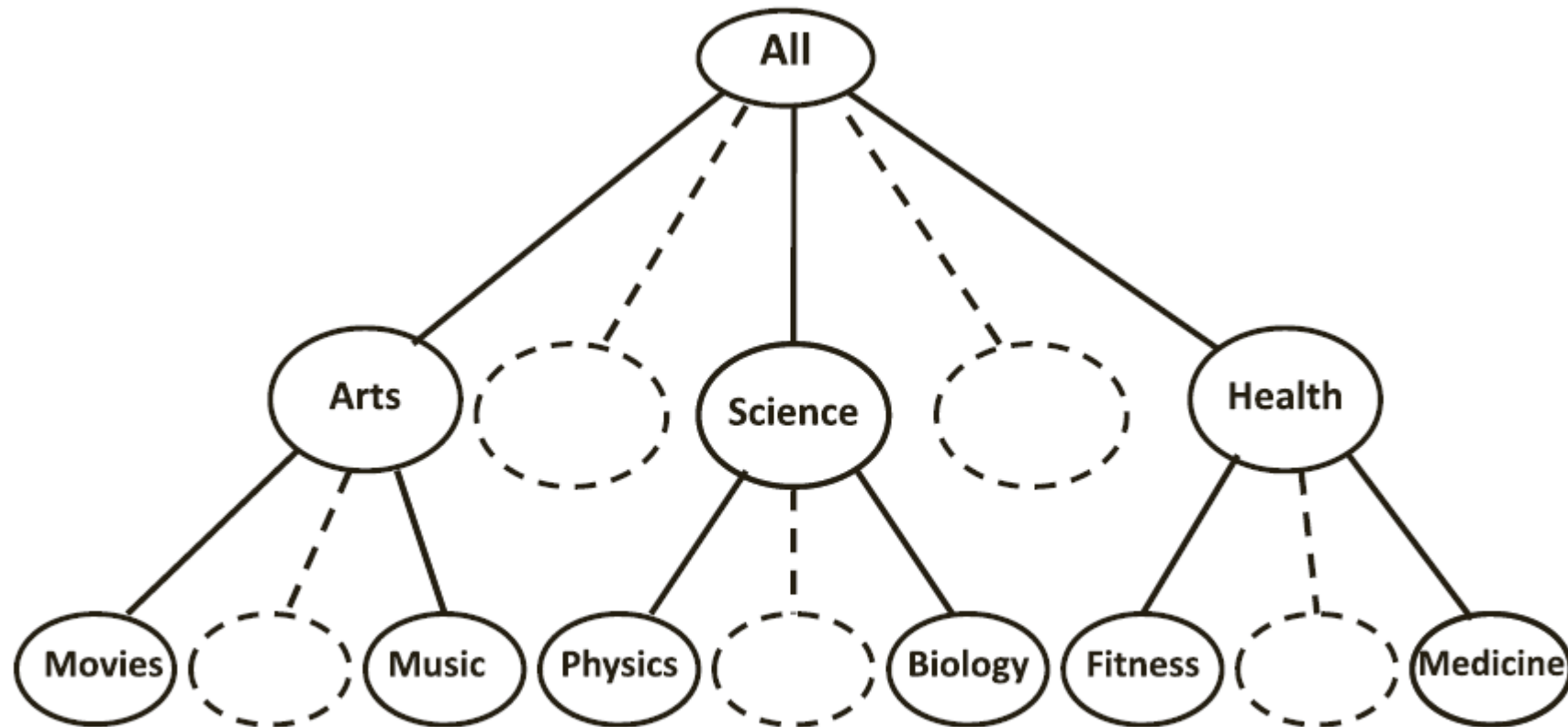


Figure 6.6: Multigranularity insights from hierarchical clustering

Hierarchical Clustering Algorithms

- **Hierarchical organization** makes manual browsing very convenient for a user, especially when the content of the clusters can be described in a semantically comprehensible way. In other cases, such hierarchical organizations can be used by indexing algorithms.
- Furthermore, such methods can sometimes also be used for creating **better “flat”** clusters. Some agglomerative hierarchical methods and divisive methods, such as bisecting k -means, can provide better quality clusters than partitioning methods such as k -means, albeit at a higher computational cost.

Hierarchical Clustering Algorithms

- There are two types of hierarchical algorithms, depending on how the hierarchical tree of clusters is constructed:
- 1. *Bottom-up (agglomerative) methods*: The individual data points are successively agglomerated into higher-level clusters. The main variation among the different methods is in the choice of objective function used to decide the **merging** of the clusters.
- 2. *Top-down (divisive) methods*: A top-down approach is used to successively partition the data points into a tree-like structure. A **flat** clustering algorithm may be used for the **partitioning** in a given step.

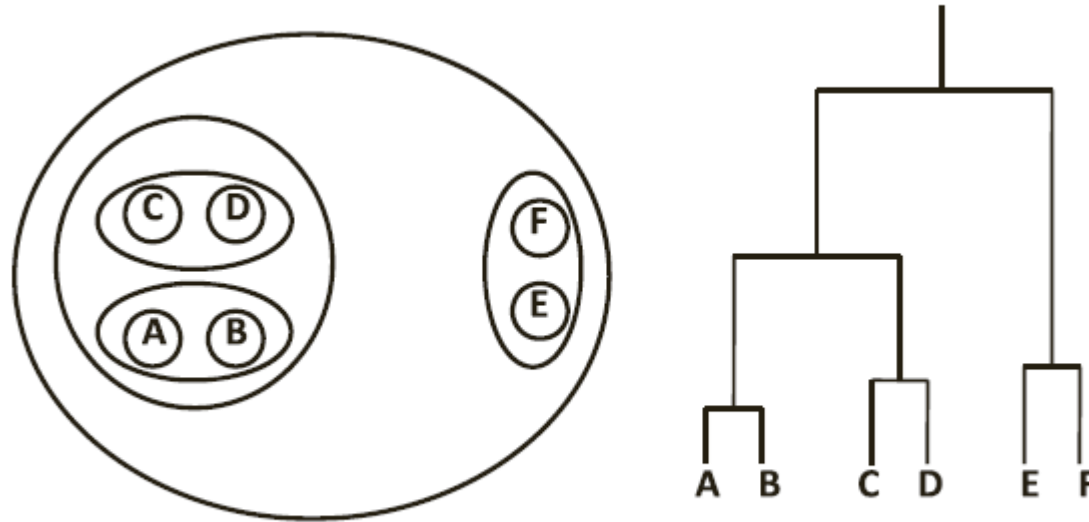
Hierarchical Clustering Algorithms

Bottom-up (agglomerative) methods

- In bottom-up methods, the data points are successively agglomerated into higher level clusters. The algorithm starts with individual data points in their own clusters and successively agglomerates them into higher level clusters. In each iteration, two clusters are selected that are deemed to be as close as possible. These clusters are merged and replaced with a newly created merged cluster. Thus, **each merging step reduces the number of clusters by 1**.
- Therefore, a method needs to be designed for **measuring proximity between clusters** containing multiple data points, so that they may be merged. It is in this choice of **computing the distances between clusters**, that most of the variations among different methods arise.

Hierarchical Clustering Algorithms

- An example of a dendrogram on successive merges on six data points, denoted by A, B, C, D, E, and F.



(a) Dendrogram

- A clustering of the data objects is obtained by cutting the dendrogram at the desired level, then each connected component forms a cluster.

Hierarchical Clustering Algorithms

- The generic agglomerative procedure with an unspecified merging criterion is illustrated in the following algorithm:

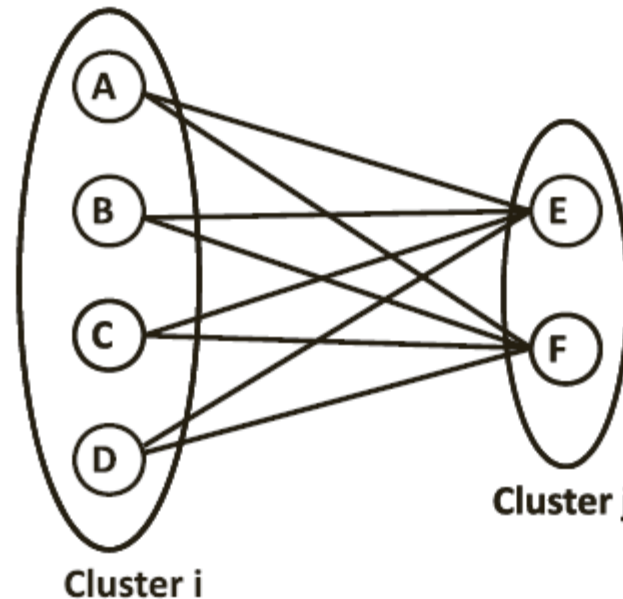
Algorithm *AgglomerativeMerge*(Data: $\mathcal{D}$)
begin
 Initialize $n \times n$ distance matrix M using $\mathcal{D}$;
 repeat
 Pick closest pair of clusters i and j using M ;
 Merge clusters i and j ;
 Delete rows/columns i and j from M and create
 a new row and column for newly merged cluster;
 Update the entries of new row and column of M ;
 until termination criterion;
 return current merged cluster set;
end

Hierarchical Clustering Algorithms

- The distances are encoded in the $n_t \times n_t$ distance matrix M . **This matrix provides the pairwise cluster distances** computed with the use of the merging criterion.
- The different choices for the merging criteria will be described later.
- The merging of two clusters corresponding to rows (columns) i and j in the matrix M requires the computation of some measure of distances between their constituent objects. For two clusters containing m_i and m_j objects, respectively, there are $m_i \cdot m_j$ pairs of distances between constituent objects.

Hierarchical Clustering Algorithms

- Example: in the figure below, there are $2 \times 4 = 8$ pairs of distances between the constituent objects, which are illustrated by the corresponding edges. The overall distance between the two clusters needs to be computed as a function of these $m_i \cdot m_j$ pairs.



(b) Group similarity computation

Hierarchical Clustering Algorithms

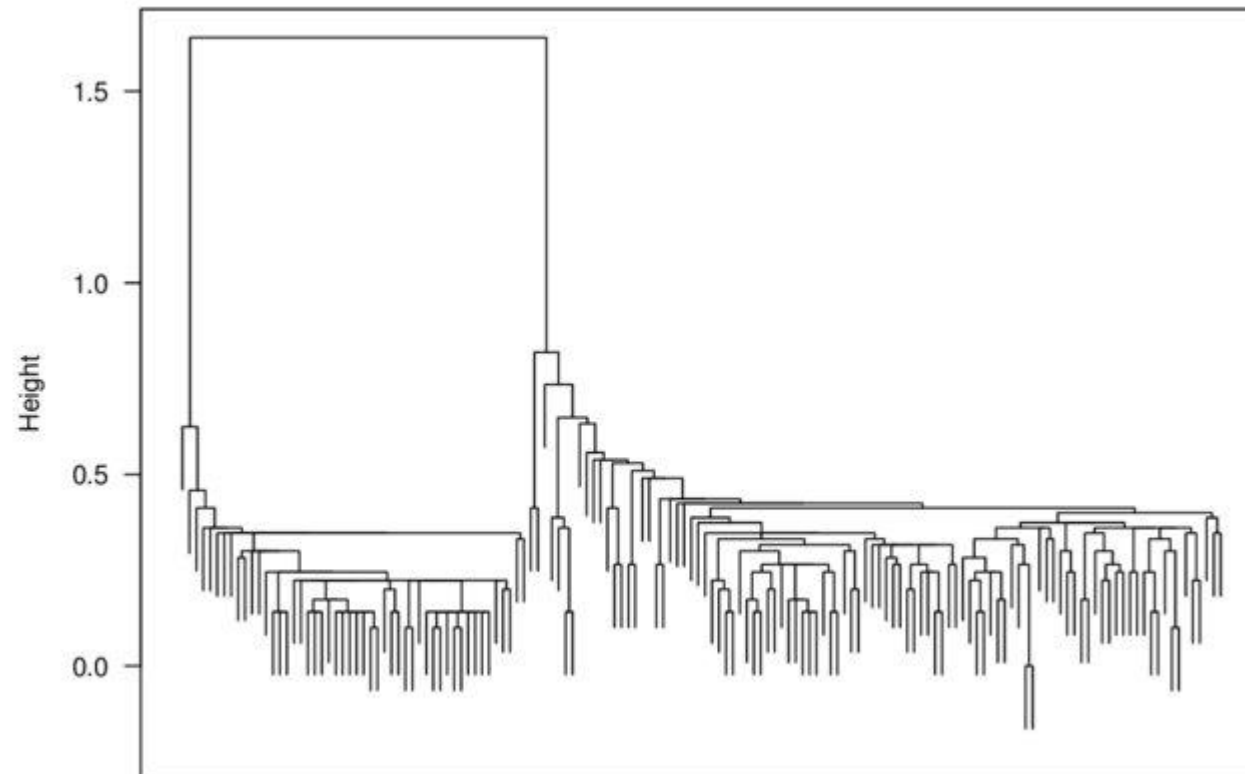
Merging criteria:

- Single linkage → take the shortest distance among the 8 distances
- Complete linkage → take the longest distance among the 8 distances
- Average linkage → calculate the average of the 8 distances

The resulting value is the one that goes into cell (i,j) of matrix M.

Hierarchical Clustering Algorithms

Dendrogram for the single linkage clustering of the Iris data set.



Hierarchical Clustering Algorithms

Top-Down Divisive Methods

- Although bottom-up agglomerative methods are typically distance-based methods, top-down hierarchical methods can be viewed as **general-purpose meta-algorithms** that can use almost any clustering algorithm as a subroutine.
- The overall approach for top-down clustering uses a **general-purpose flat-clustering algorithm** A as a subroutine. The algorithm initializes the tree at the root node containing all the data points. In each iteration, the data set at a particular node of the current tree is split into multiple nodes (clusters).

Hierarchical Clustering Algorithms

Algorithm *GenericTopDownClustering*(Data: $\mathcal{D}$, Flat Algorithm: $\mathcal{A}$)
begin
 Initialize tree $\mathcal{T}$ to root containing $\mathcal{D}$;
 repeat
 Select a leaf node L in $\mathcal{T}$ based on pre-defined criterion;
 Use algorithm $\mathcal{A}$ to split L into $L_1 \dots L_k$;
 Add $L_1 \dots L_k$ as children of L in $\mathcal{T}$;
 until termination criterion;
end

Figure 6.10: Generic top-down meta-algorithm for clustering

Hierarchical Clustering Algorithms

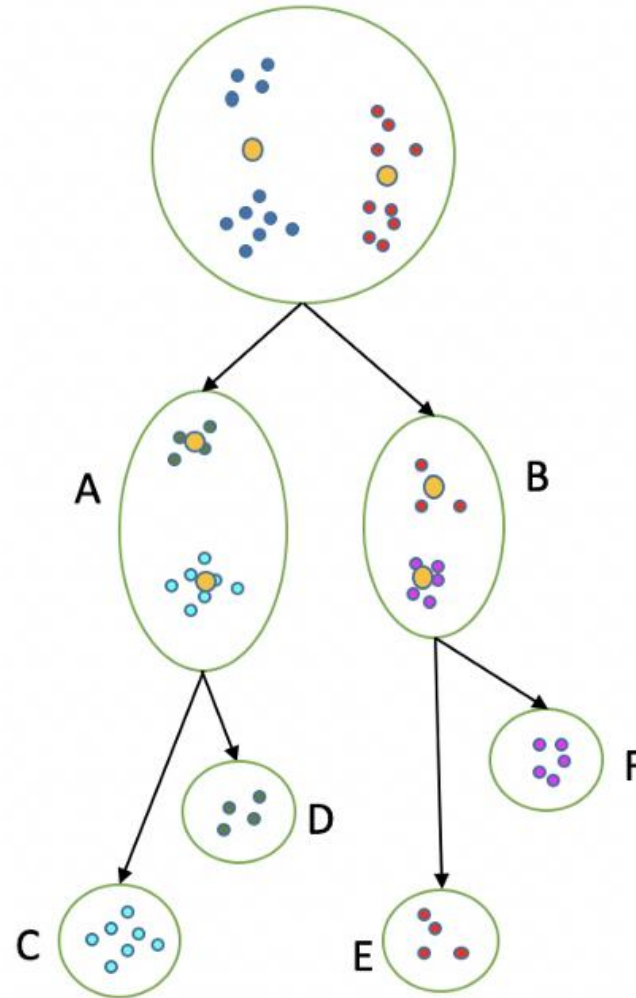
- The algorithm recursively splits nodes with a top-down approach until either a certain height of the tree is achieved or each node contains fewer than a predefined number of data objects. A wide variety of algorithms can be designed with different instantiations of the algorithm $\mathcal{A}$ and growth strategy. Note that the algorithm $\mathcal{A}$ can be any arbitrary clustering algorithm, and not just a distance-based algorithm.

Hierarchical Clustering Algorithms

Bisecting *k*-Means

- The bisecting *k*-means algorithm is a top-down hierarchical clustering algorithm in which each node is split into exactly **two children** with a **2-means algorithm**. To split a node into two children, several **randomized trial runs** of the split are used, and the split that has the best impact on the overall clustering objective is used. Several variants of this approach use different growth **strategies for selecting the node to be split**.
- For example, **the heaviest node** may be split first, or the node with **the smallest distance from the root** may be split first.
- These different choices lead to balancing either the cluster weights or the tree height.

Hierarchical Clustering Algorithms

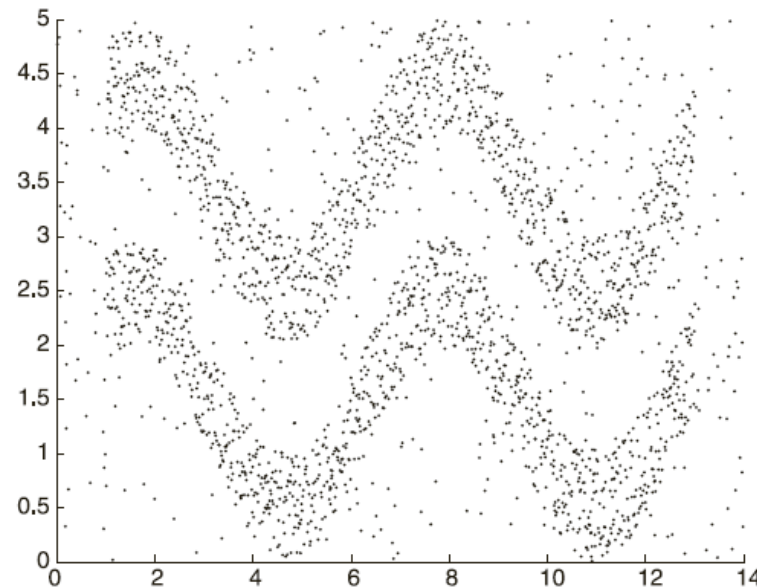


Grid/Density-Based Algorithms

- One of the major problems with distance-based methods is that the shape of the underlying clusters is already defined implicitly by the underlying distance function or probability distribution.
- For example, a k -means algorithm implicitly assumes a spherical shape for the cluster. In practice, the clusters may be hard to model with a prototypical shape implied by a distance function or probability distribution.

Grid/Density-Based Algorithms

- To understand this point, consider the clusters illustrated below. It is evident that there are two clusters of sinusoidal shape in the data. However, virtually any choice of representatives in a k -means algorithm will result in the representatives of one of the clusters pulling away data points from the other.



(a) Arbitrarily-shaped clusters

Grid/Density-Based Algorithms

- **Density-based algorithms** are very helpful in such scenarios. The core idea in such algorithms is to first identify **fine-grained dense regions in the data**. These form the “**building blocks**” for constructing the arbitrarily-shaped clusters. These can also be considered **pseudo-data points** that need to be **re-clustered** together carefully into groups of arbitrary shape. Thus, most density-based methods can be considered **two-level hierarchical algorithms**.

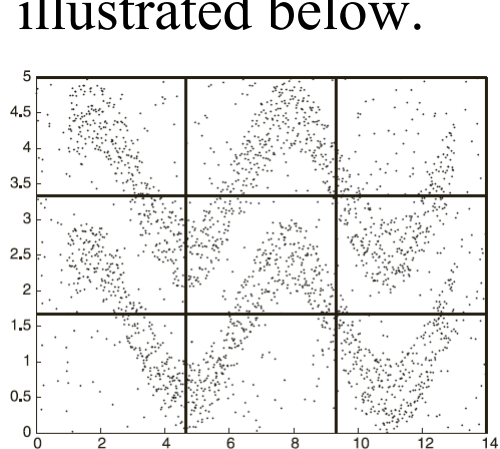
Grid/Density-Based Algorithms

- Many variations of this broader principle exist, depending on the particular type of building blocks that are chosen.
- For example, in grid-based methods, the fine-grained clusters are grid-like *regions* in the data space.
- When pre-selected data points in dense regions are clustered with a single-linkage method, the approach is referred to as *DBSCAN*.
- Other more sophisticated density-based methods, such as *DENCLUE*, use gradient descent on the kernel-density estimates to create the building blocks.

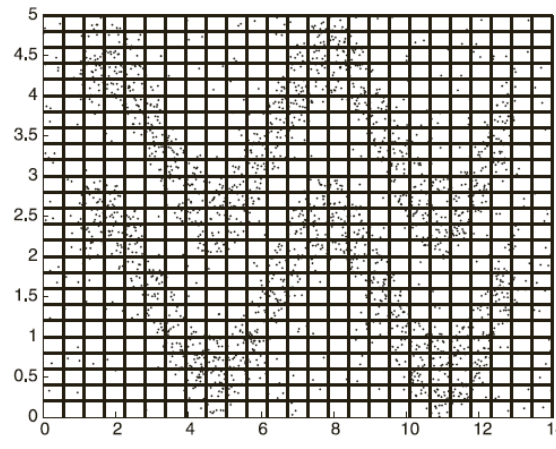
Grid/Density-Based Algorithms

Grid-based methods

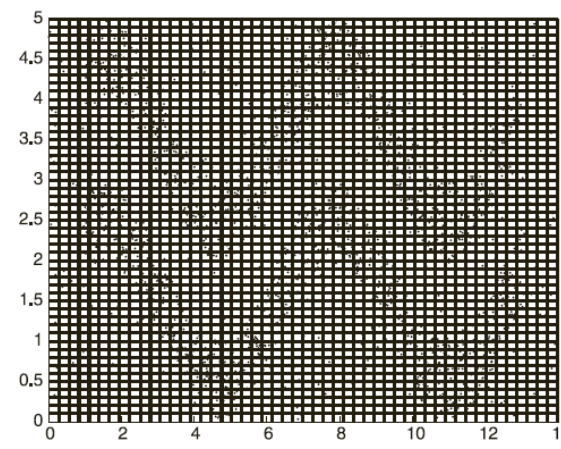
- In this technique, the data is discretized into p intervals that are typically equi-width intervals. Other variations such as equi-depth intervals are possible, though they are often not used **in order to retain the intuitive notion of density**. For a d -dimensional data set, this leads to a total of p^d **hyper-cubes** in the underlying data.
- Examples of grids of different granularity with $p = 3, 25$, and 80 are illustrated below.



(b) Rough-grained grid



(c) Moderate-grained grid



(d) Fine-grained grid

Grid/Density-Based Algorithms

- The resulting hyper-cubes (**rectangles** in the previous figures) are the building blocks in terms of which the clustering is defined. A density threshold τ is used to determine the subset of the p^d hyper-cubes that are **dense**.
- In most real data sets, an arbitrarily shaped cluster will result in multiple dense regions that are **connected together by a side or at least a corner**:
 - Two grid regions are said to be **adjacently connected**, if they share a side in common.
 - A weaker version of this definition considers two regions to be adjacently connected if they share a **corner** in common.

Many grid-clustering algorithms use the strong definition of adjacent connectivity, where a side is used instead of a corner.

In general, for data points in d -dimensional space, two d -dimensional cubes may be defined as adjacent, if **they have share a surface of dimensionality at least r** , for some user-defined parameter $r < d$.

Grid/Density-Based Algorithms

- This **directly adjacent connectivity** can be generalized to **indirect density connectivity** between grid regions that are not immediately adjacent to one another.
- Two grid regions are **density connected**, if a path can be found from one grid to the other containing only a sequence of adjacently connected grid regions.
- The **goal of grid-based clustering is to determine the connected (directly or indirectly) regions created by such grid cells.**
- It is easy to determine such connected grid regions by using **a graph-based model on the grids**. Each **dense grid node** is associated with a node in the graph, and each edge represents adjacent **connectivity**.
- The **connected components in the graph** may be determined by using breadth-first or depth-first traversal on the graph, starting from nodes in different components.
- **The data points in these connected components are reported as the final clusters.**

Grid/Density-Based Algorithms

Example of the construction of the clusters of arbitrary shape from the building blocks is illustrated in this figure.

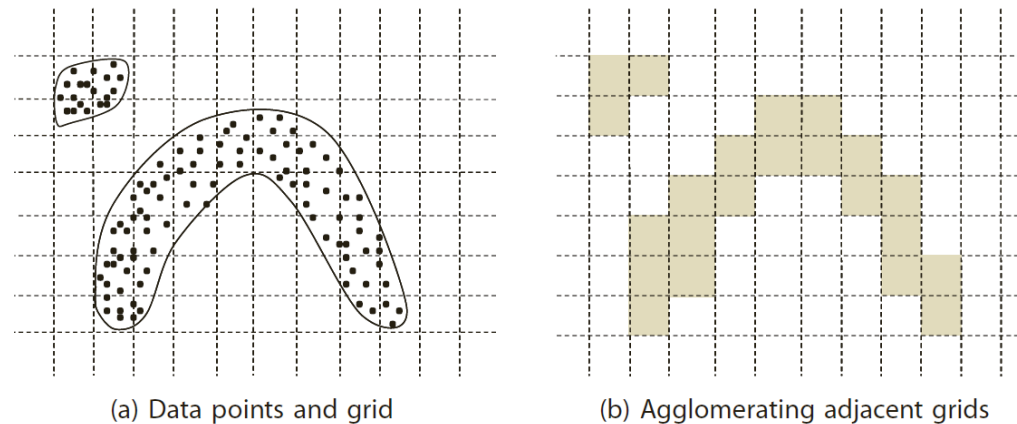


Figure 6.13: Agglomerating adjacent grids

Note that the corners of the clusters found are artificially rectangular, which is one of the limitations of grid-based methods. The generic pseudocode for the grid-based approach is the following:

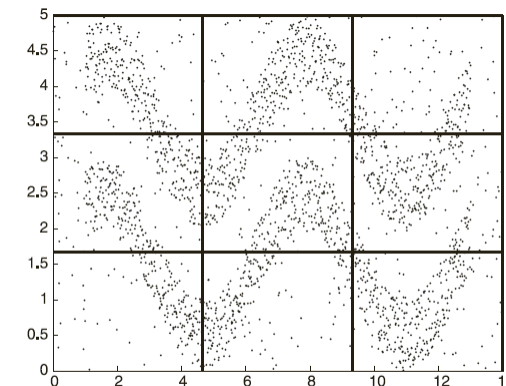
Grid/Density-Based Algorithms

Algorithm *GenericGrid*(Data: $\mathcal{D}$, Ranges: p , Density: τ)
begin
 Discretize each dimension of data $\mathcal{D}$ into p ranges;
 Determine dense grid cells at density level τ ;
 Create graph in which dense grids are connected if they are adjacent;
 Determine connected components of graph;
 return points in each connected component as a cluster;
end

- One desirable property of grid-based (and most other density-based) algorithms is that **the number of data clusters is not pre-defined in advance**, as in k -means algorithms. Rather, the goal is to return the natural clusters in the data together with their corresponding shapes.

Grid/Density-Based Algorithms

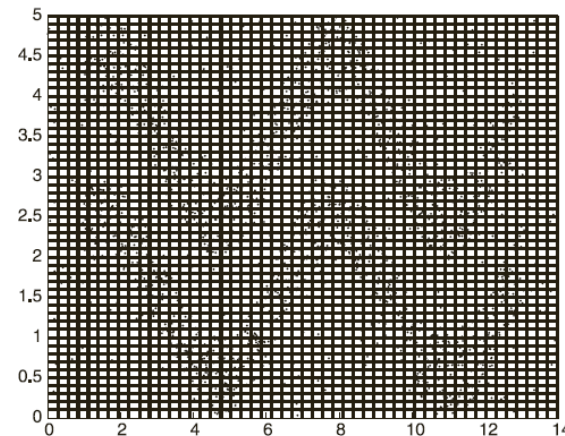
- On the other hand, two different parameters need to be defined corresponding to the **number of grid ranges p** and the **density threshold τ** . The correct choice of these parameters is often difficult and semantically un-intuitive to guess.
- An inaccurate choice can lead to unintended consequences:
 - When the number of grid ranges selected is **too small**, the data points from multiple clusters will be present in the same grid region. This will result in the undesirable merging of clusters.



(b) Rough-grained grid

Grid/Density-Based Algorithms

- When the number of grid ranges selected is **too large**, this **will result in many empty grid cells even within the clusters**. As a result, natural clusters in the data may be disconnected by the algorithm. A larger number of grid ranges also leads to **computational challenges** because of the increasing number of grid cells.



(d) Fine-grained grid

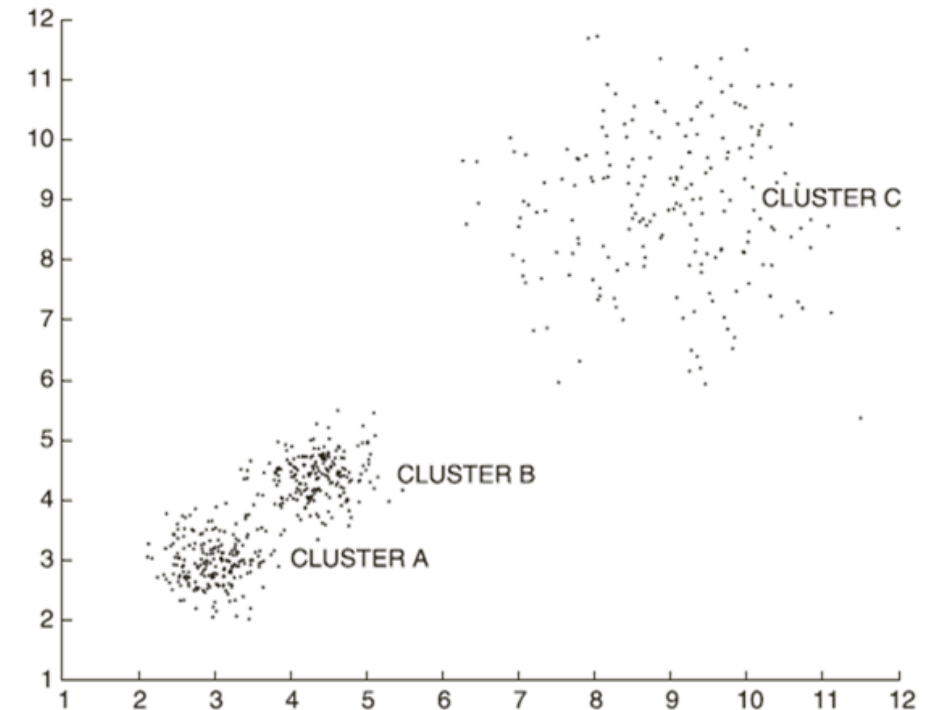
Grid/Density-Based Algorithms

2. The choice of the density threshold τ has a similar effect on the clustering.
 - when the density threshold τ is too low, all clusters, including the ambient noise, will be merged into a single large cluster; on the other hand,
 - an unnecessarily high density threshold τ can partially or entirely miss a cluster.
- The two drawbacks discussed above are serious ones, especially when there are significant variations in the cluster size and density over different local regions.

Grid/Density-Based Algorithms

Practical issues

- Grid-based methods do not require the specification of the number of clusters, and also do not assume any specific shape for the clusters. However, this comes at the expense of having to specify a **density parameter** τ , which is not always **intuitive** from an analytical perspective. The choice of **grid resolution** is also challenging because it is not clear how it can be related to the density τ . As will be evident later, this is much easier with *DBSCAN*, where the resolution of the density-based approach is more easily related to the specified density threshold.
- A major challenge with many density-based methods, including grid-based methods, is that they use a single density parameter τ globally. However, the clusters in the underlying data may have varying density, as illustrated below.



Grid/Density-Based Algorithms



- In this particular case, if the density threshold is selected to be too high, then cluster C may be missed. On the other hand, if the density threshold is selected to be too low, then clusters A and B may be merged artificially.
- In such cases, distance-based algorithms, such as k -means, may be more effective than a density-based approach. **This problem is not specific to the grid-based method but is generally encountered by all density-based methods.**

Grid/Density-Based Algorithms

- The use of rectangular grid regions is an approximation of this class of methods. This approximation degrades with increasing dimensionality because high-dimensional rectangular regions are poor approximations of the underlying clusters.
- Furthermore, grid-based methods become computationally infeasible in high dimensions because the number of grid cells increase exponentially with the underlying data dimensionality.

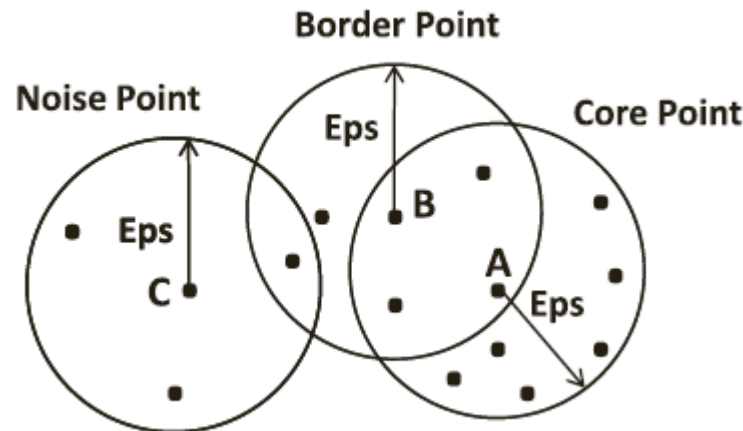
Grid/Density-Based Algorithms

DBSCAN

- The *DBSCAN* approach works on a very similar principle as grid-based methods. However, unlike grid-based methods, the density characteristics of data points are used to merge them into clusters. Therefore, the individual data points in dense regions are used as building blocks after classifying them on the basis of their density.
- The **density** of a data point is defined by the number of points that lie within a radius *Eps* of that point (including the point itself). The densities of these spherical regions are used to classify the data points into *core*, *border*, or *noise points*. These notions are defined as follows:

Grid/Density-Based Algorithms

1. *Core point*: A data point is defined as a core point, if it contains at least τ data points.
2. *Border point*: A data point is defined as a border point, if it contains less than τ points, but it also contains at least one core point within a radius Eps .
3. *Noise point*: A data point that is neither a core point nor a border point is defined as a *noise point*.



$$\tau = 10$$

Grid/Density-Based Algorithms

- After the **core**, **border**, and **noise** points have been determined, the *DBSCAN* clustering algorithm proceeds as follows.
- First, a **connectivity graph** is constructed with respect to the **core points**, in which each node corresponds to a core point, and an edge is added between a pair of core points, if and only if they are within a distance of *Eps* from one another.
 - Note that the graph is constructed on the data *points* rather than on partitioned *regions*, as in grid-based algorithms.
- Then, **all connected components of this graph are identified**. These correspond to the clusters constructed on the core points.
- Finally, **border points are then assigned to the cluster with which they have the highest level of connectivity**. The resulting groups are reported as clusters and noise points are reported as outliers.

Grid/Density-Based Algorithms

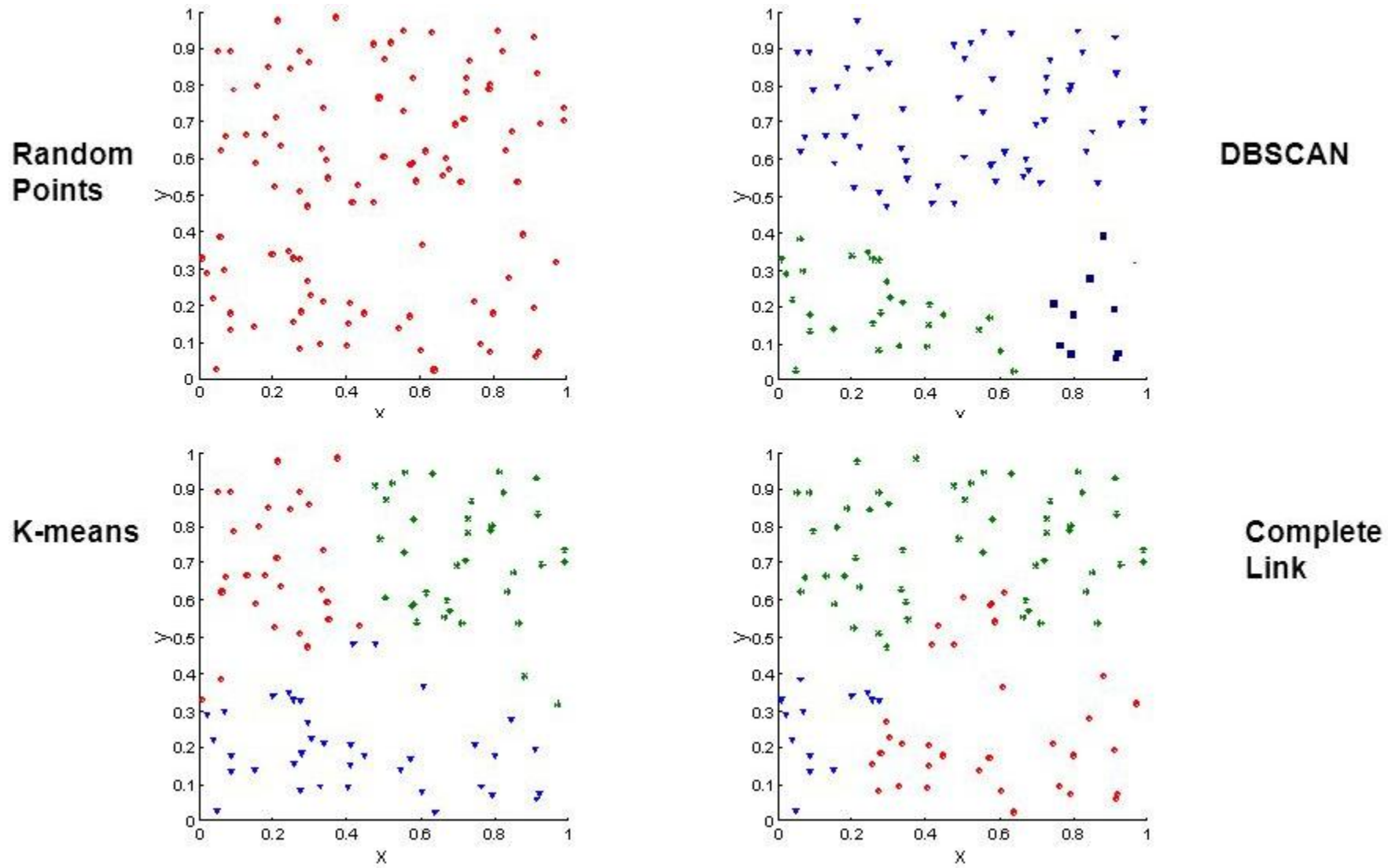
Algorithm *DBSCAN*(Data: $\mathcal{D}$, Radius: Eps , Density: τ)
begin
 Determine core, border and noise points of $\mathcal{D}$ at level (Eps, τ) ;
 Create graph in which core points are connected
 if they are within Eps of one another;
 Determine connected components in graph;
 Assign each border point to connected component
 with which it is best connected;
 return points in each connected component as a cluster;
end

Cluster Validation

- After a clustering of the data has been determined, it is important to evaluate its quality. This problem is referred to as *cluster validation*.
- But “*clusters are in the eye of the beholder*”!
- Then why do we want to evaluate them?
 - To avoid finding patterns in noise
 - To compare clustering algorithms
 - To compare two sets of clusters
 - To compare two clusters

Cluster Validation

Clusters found in random data



Cluster Validation

- Cluster validation is often difficult in real data sets because the problem is defined in an unsupervised way. Therefore, no external validation criteria may be available to evaluate a clustering.
- Thus, a number of *internal* criteria may be defined to validate the quality of a clustering. The major problem with internal criteria is that *they may be biased toward one algorithm or the other*, depending on how they are defined.
- In some cases, *external validation criteria* may be available when a test data set is synthetically generated, and therefore the true (ground-truth) clusters are known. Alternatively, for real data sets, the class labels, if available, may be used as proxies for the cluster identifiers. In such cases, the evaluation is more effective.

Cluster Validation

Internal Validation Criteria

- Internal validation criteria are used when no external criteria are available to evaluate the quality of a clustering. In most cases, **the criteria used to validate the quality of the algorithm are borrowed directly from the objective function**, which is optimized by a particular clustering model.
- For example, virtually any of the objective functions in the k -representatives, and agglomerative methods could be used for validation purposes.
- The problem with the use of these criteria is obvious in comparing algorithms with disparate methodologies. **A validation criterion will always favor a clustering algorithm that uses a similar kind of objective function for its optimization.** Nevertheless, in the absence of external validation criteria, this is the best that one can hope to achieve. **Such criteria can be effective in comparing two algorithms using the same broad approach.**
- The commonly used internal evaluation criteria are as follows:

Cluster Validation

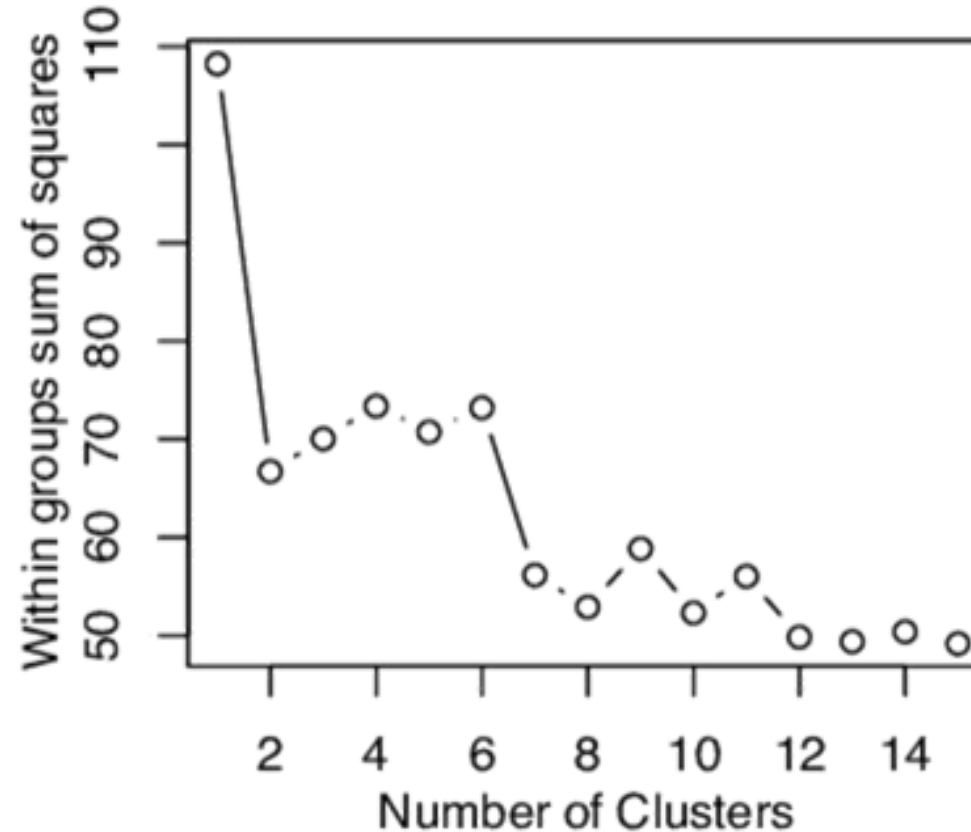
1. *Sum of square distances to centroids*: In this case, the centroids of the different clusters are determined, and the sum of squared (SSQ) distances are reported as the corresponding objective function.

Smaller values of this measure are indicative of better cluster quality.

This measure is obviously more optimized to distance-based algorithms, such as *k*-means, as opposed to a density-based method, such as *DBSCAN*.

Another problem with SSQ is that the **absolute distances provide no meaningful information** to the user about the quality of the underlying clusters.

Cluster Validation



Total sum of squares within clusters vs # clusters.

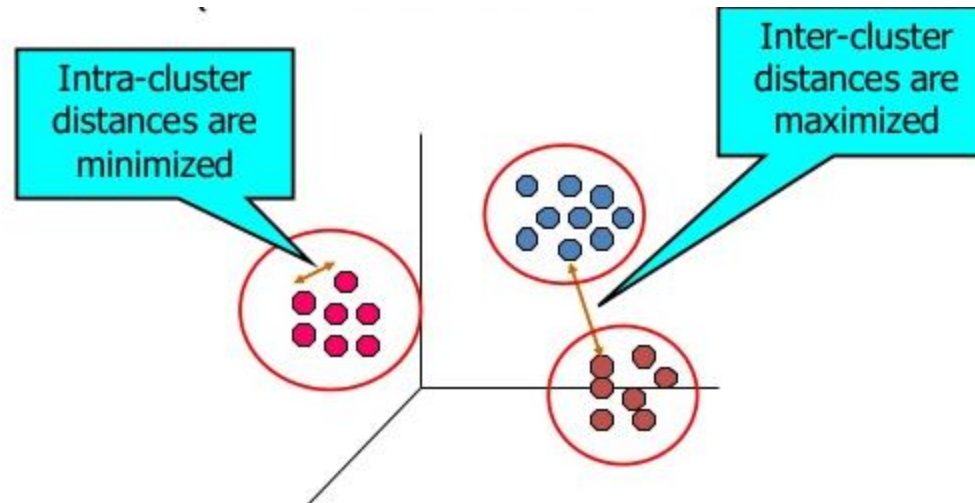
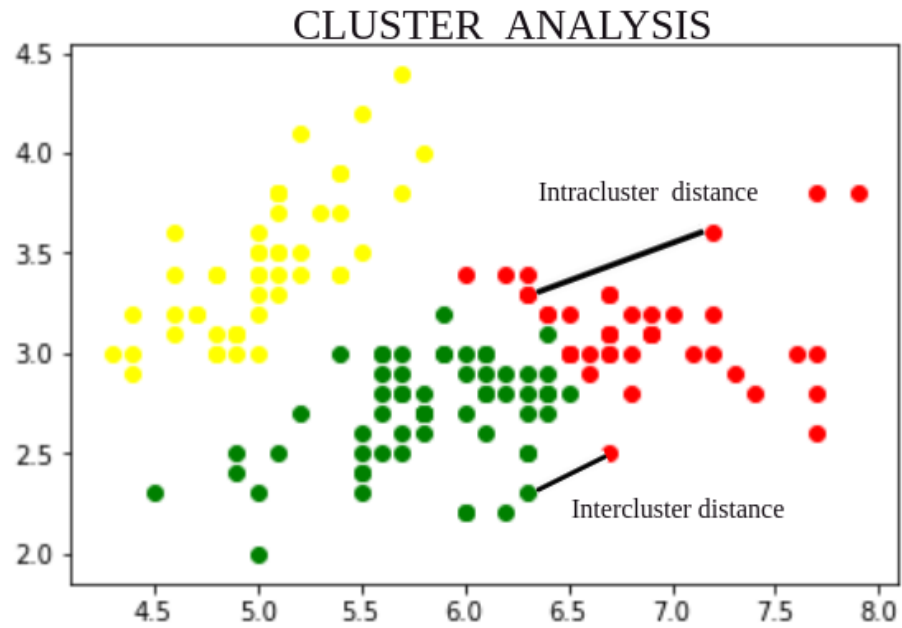
Cluster Validation

2. *Intracluster to intercluster distance ratio*: This measure is more detailed than the SSQ measure. The idea is to sample r pairs of data points from the underlying data. Of these, let P be the set of pairs that belong to the same cluster found by the algorithm. The remaining pairs are denoted by set Q . The average intercluster distance and intracluster distance are defined as follows:

$$Intra = \sum_{(\overline{X_i}, \overline{X_j}) \in P} dist(\overline{X_i}, \overline{X_j}) / |P|$$
$$Inter = \sum_{(\overline{X_i}, \overline{X_j}) \in Q} dist(\overline{X_i}, \overline{X_j}) / |Q|.$$

Then the ratio of the average intracluster distance to the intercluster distance is given by *Intra/Inter*. Small values of this measure indicate better clustering behavior.

Cluster Validation



Cluster Validation

3. *Silhouette coefficient*:

Let $D_{avg}^{in}_i$

be the average distance of $\underline{X}_i$ to data points within the cluster of $\underline{X}_i$.

The average distance of data point $\underline{X}_i$ to the points in each cluster (other than its own) is also computed.

Let $D_{min}^{out}_i$

represent the minimum of these (average) distances, over the other clusters.

Cluster Validation

- Then, the **silhouette coefficient** S_i specific to the i -th object, is as follows:

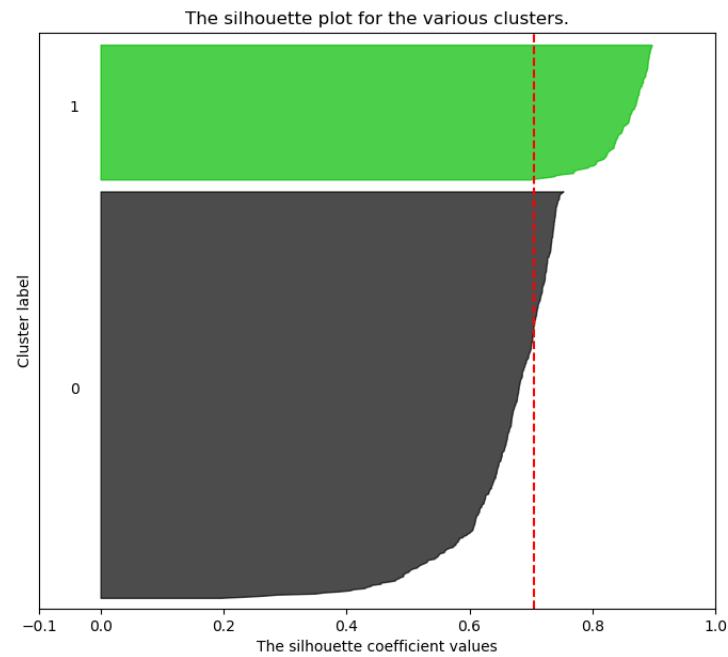
$$S_i = \frac{Dmin_i^{out} - Davg_i^{in}}{\max\{Dmin_i^{out}, Davg_i^{in}\}}$$

- The **overall** silhouette coefficient is the average of the data point-specific coefficients.
- The silhouette coefficient will be drawn from the range $(-1, 1)$.
- Large positive values indicate highly separated clustering, and negative values are indicative of some level of “mixing” of data points from different clusters.
- This is because $Dmin_i^{out}$ will be less than $Davg_i^{in}$ only in cases where data point $\underline{X}_i$ is closer to at least one other cluster than its own cluster.

One advantage of this coefficient is that the absolute values provide a good intuitive feel of the quality of the clustering.

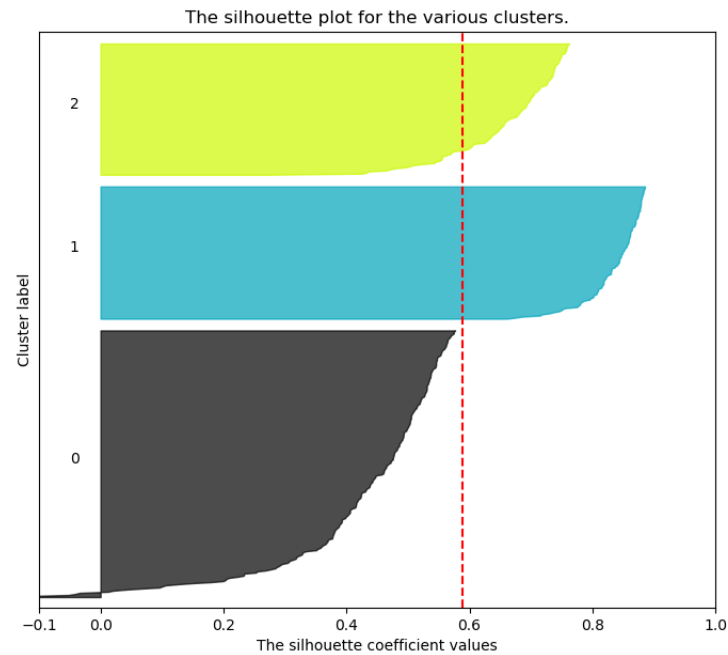
Cluster Validation

Silhouette analysis for KMeans clustering on sample data with $n_clusters = 2$



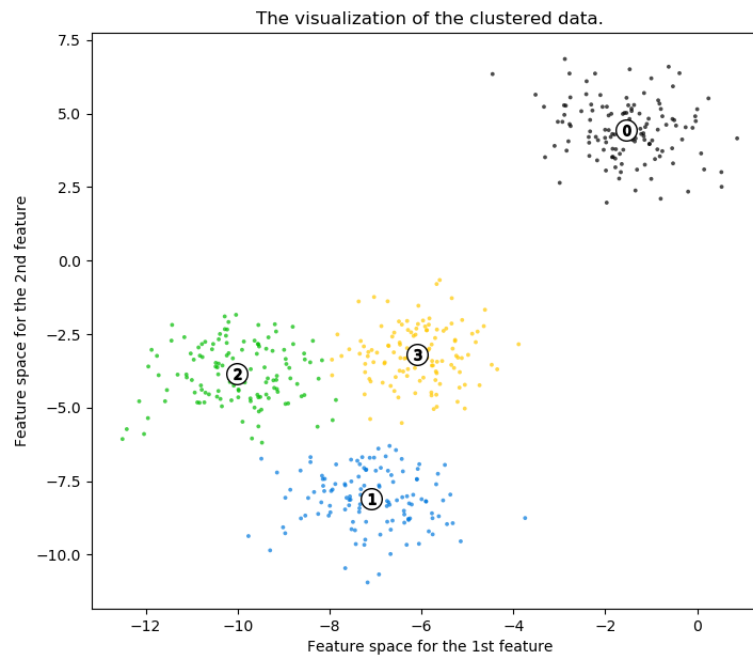
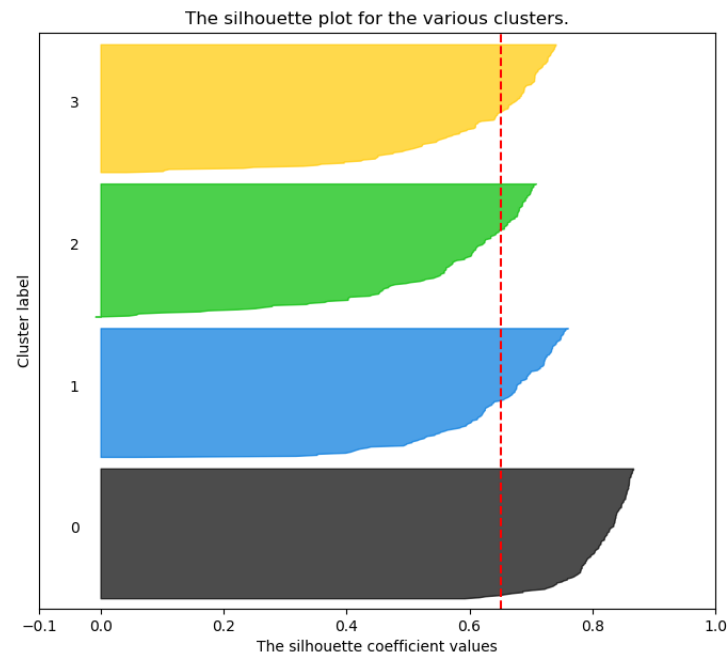
Cluster Validation

Silhouette analysis for KMeans clustering on sample data with $n_clusters = 3$



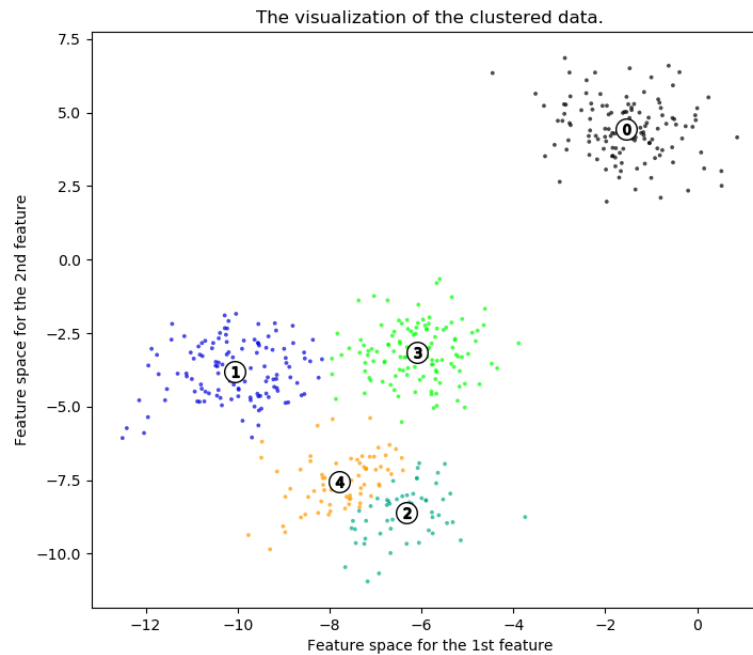
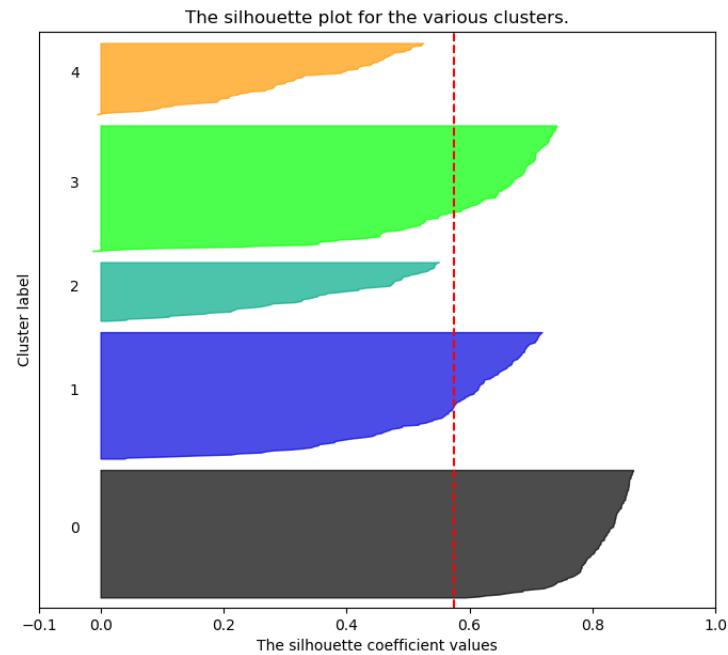
Cluster Validation

Silhouette analysis for KMeans clustering on sample data with $n_clusters = 4$



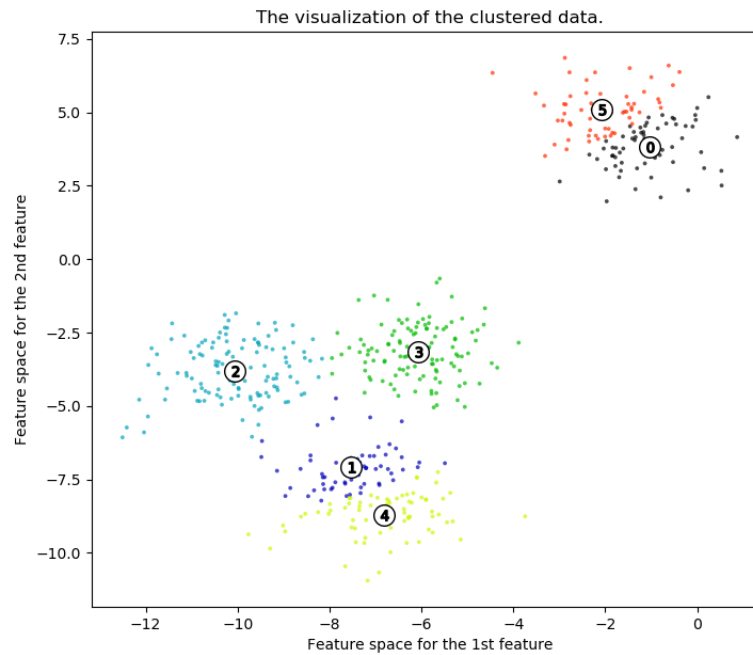
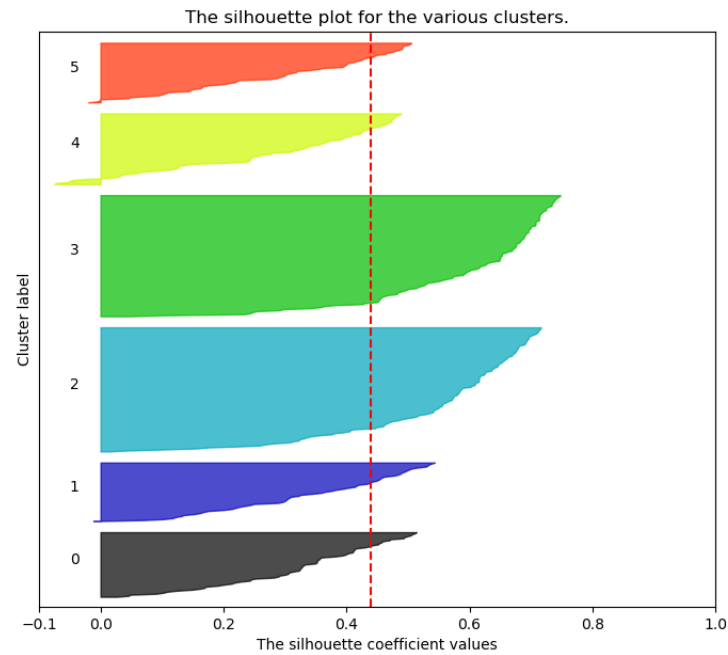
Cluster Validation

Silhouette analysis for KMeans clustering on sample data with $n_clusters = 5$



Cluster Validation

Silhouette analysis for KMeans clustering on sample data with n_clusters = 6



Bibliographic references

Charu C. Aggarwal

Data Mining

Springer 2015

Chapter 6

Sections 6.1 – 6.6, 6.9

